Machine Intelligence 1 Tutorial

03 Multilayer Perceptrons and the Backpropagation algorithm

Fachgebiet Neuronale Informationsverarbeitung (NI)

WiSe 25/26

Our agenda for today

Limitations of Perceptrons
- Solving the XOR problem

Feedforward MLP
- Navigating through the indices

Training a neural network
- Recap of gradient-based learning
- Computing gradients for an MLP

The Backpropagation algorithm
- Forward and backward propagation
- Example: MLP with fully connected layers
- Summary of the Backpropagation for gradient descent

Symmetries
- Permutation of indices within consecutive layers
- Sign-reversal across layers

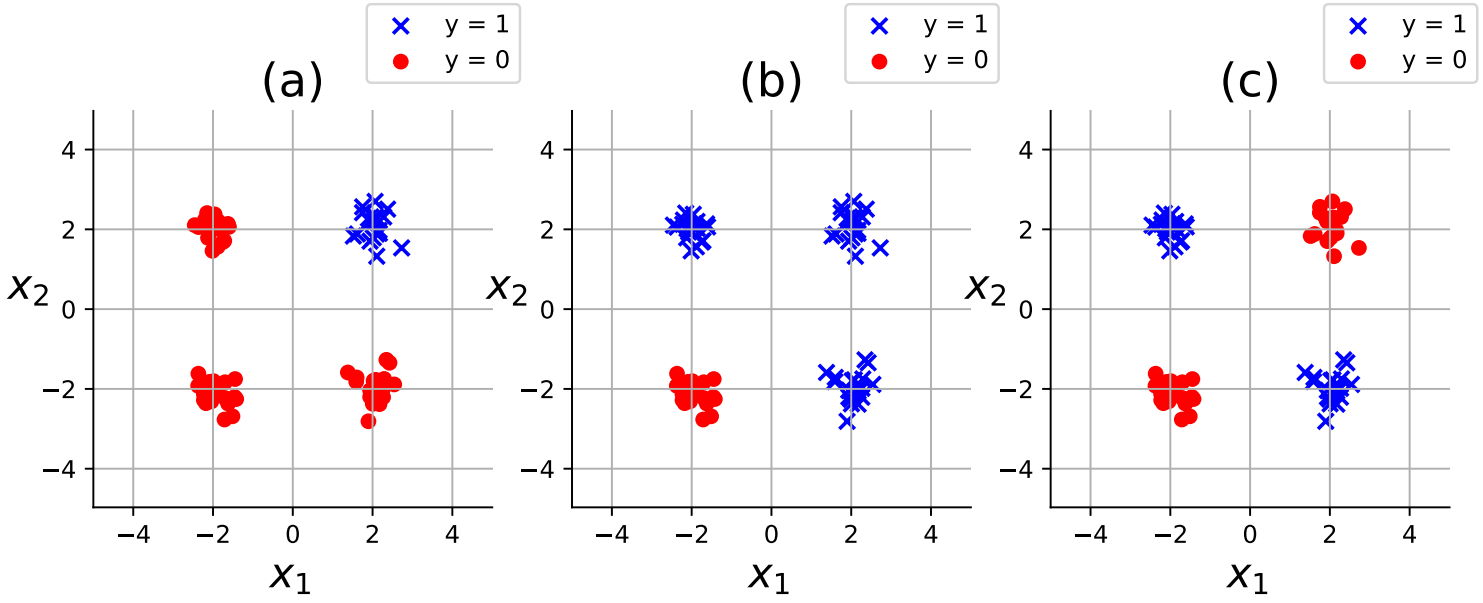
Limitations of Perceptrons

From neurons to neural networks

Linear classifiers = linear decision boundaries

Consider the following binary classification problems with $\underline{\mathbf{x}} \in \mathbb{R}^2$.

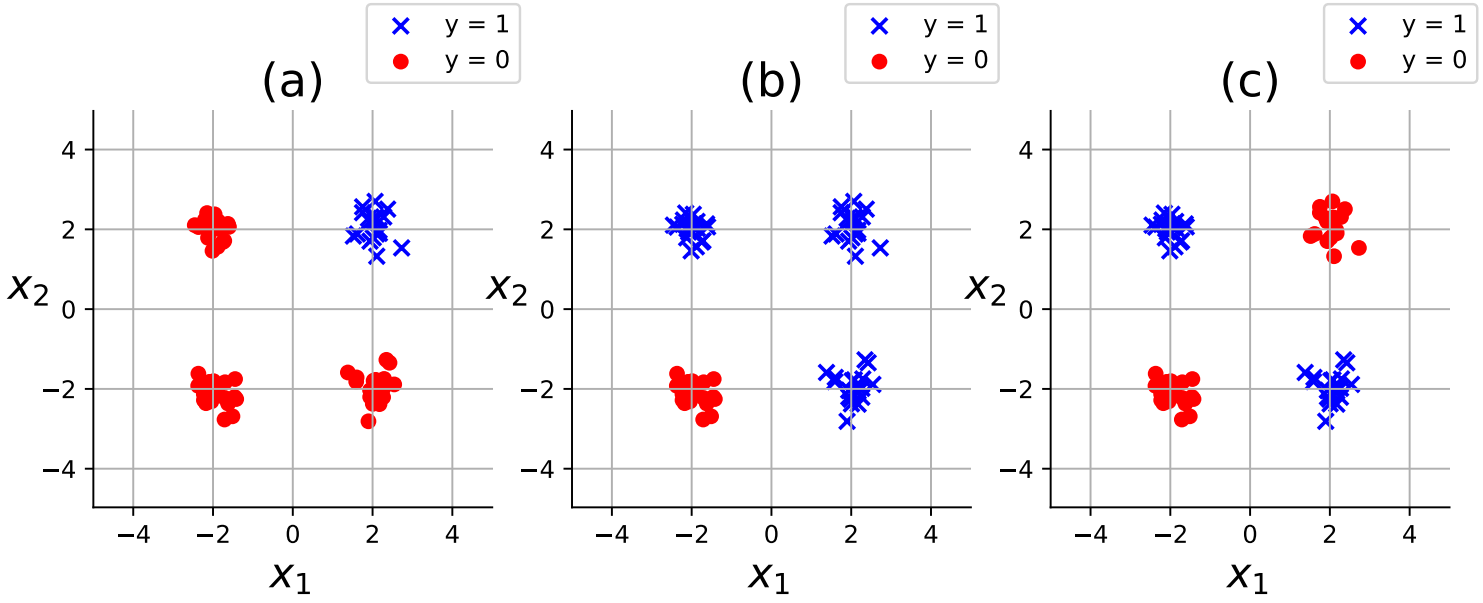
Q: *Can you find a line that separates the two classes for each case?*



Linear classifiers = linear decision boundaries

Consider the following binary classification problems with $\underline{\mathbf{x}} \in \mathbb{R}^2$.

Q: *Can you find a line that separates the two classes for each case?*

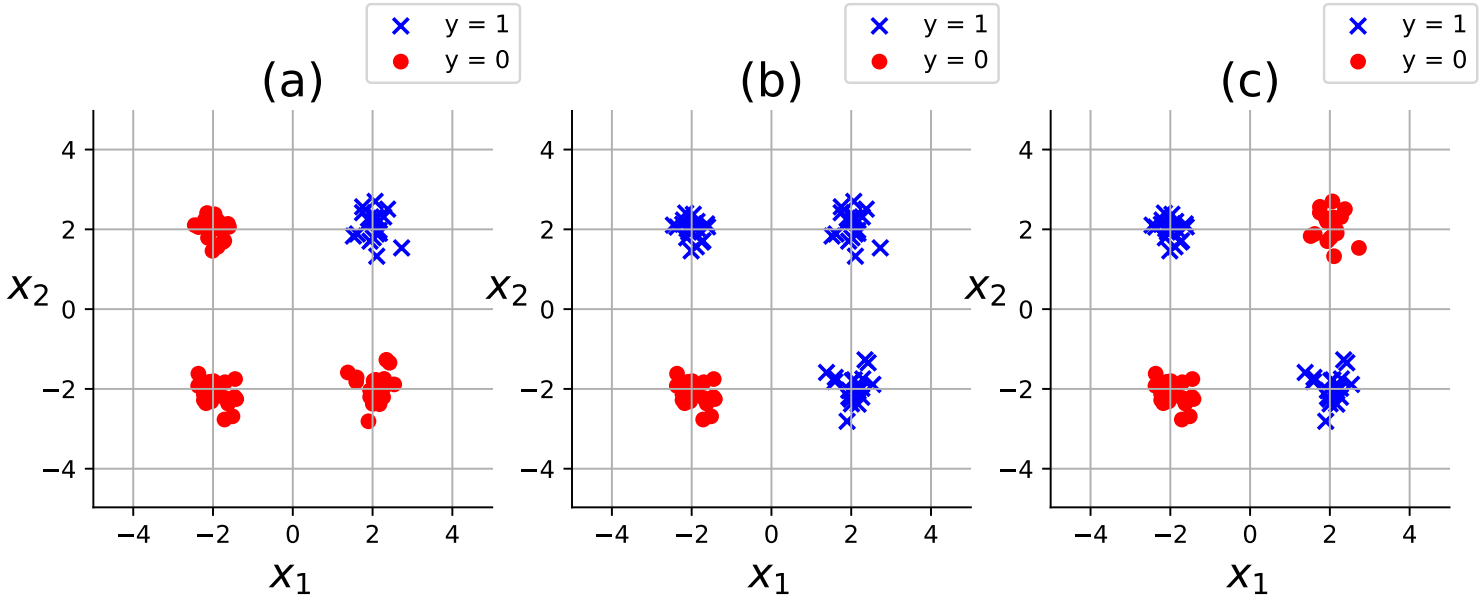


Q: *Can we solve the XOR problem with multiple perceptrons? How?*

Linear classifiers = linear decision boundaries

Consider the following binary classification problems with $\underline{\mathbf{x}} \in \mathbb{R}^2$.

Q: *Can you find a line that separates the two classes for each case?*



Q: *Can we solve the XOR problem with multiple perceptrons? How?*

$$\mathrm{XOR}(x_1, x_2) = (\, x_1 \text{ AND } \overline{x}_2 \,) \;\; \text{OR} \;\; (\, \overline{x}_1 \text{ AND } x_2 \,)$$

Inversion/NOT-Gate

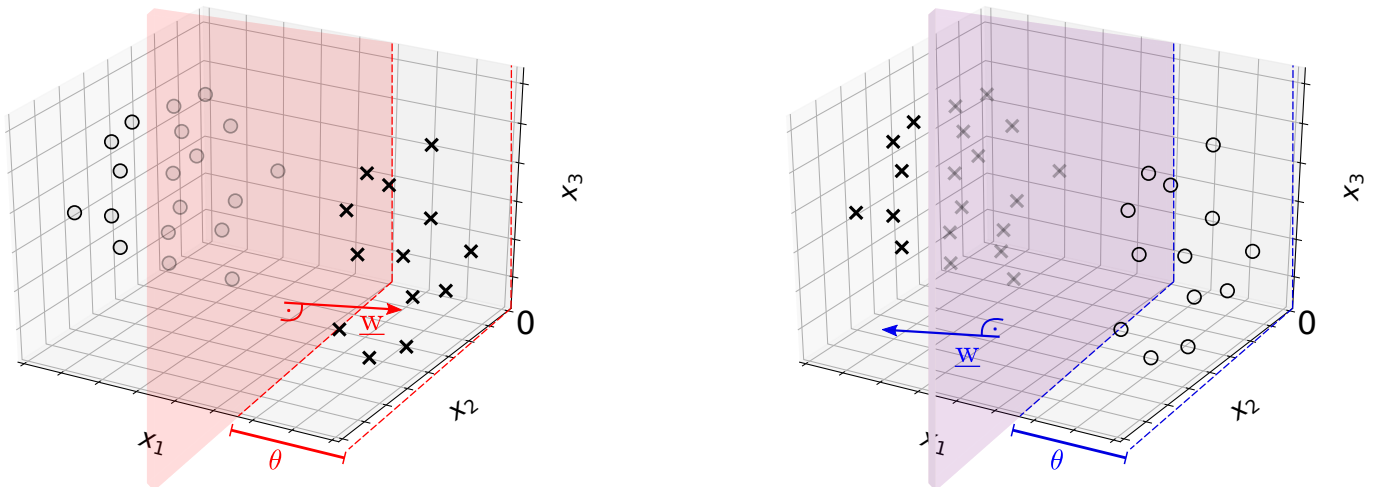


Figure: Inverting a neuron's response by flipping the hyperplane

Solving the XOR problem

$$\mathrm{XOR}(x_1, x_2) = (\, x_1 \text{ AND } \overline{x}_2 \,) \;\; \text{OR} \;\; (\, \overline{x}_1 \text{ AND } x_2 \,)$$

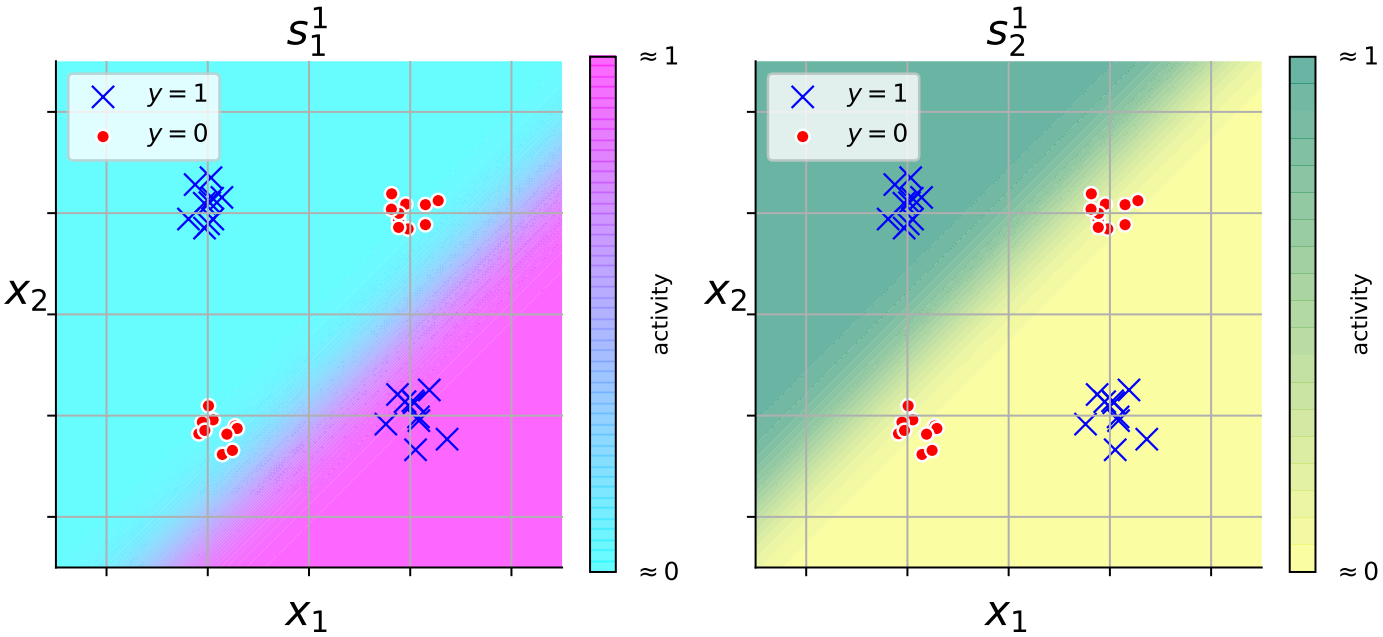
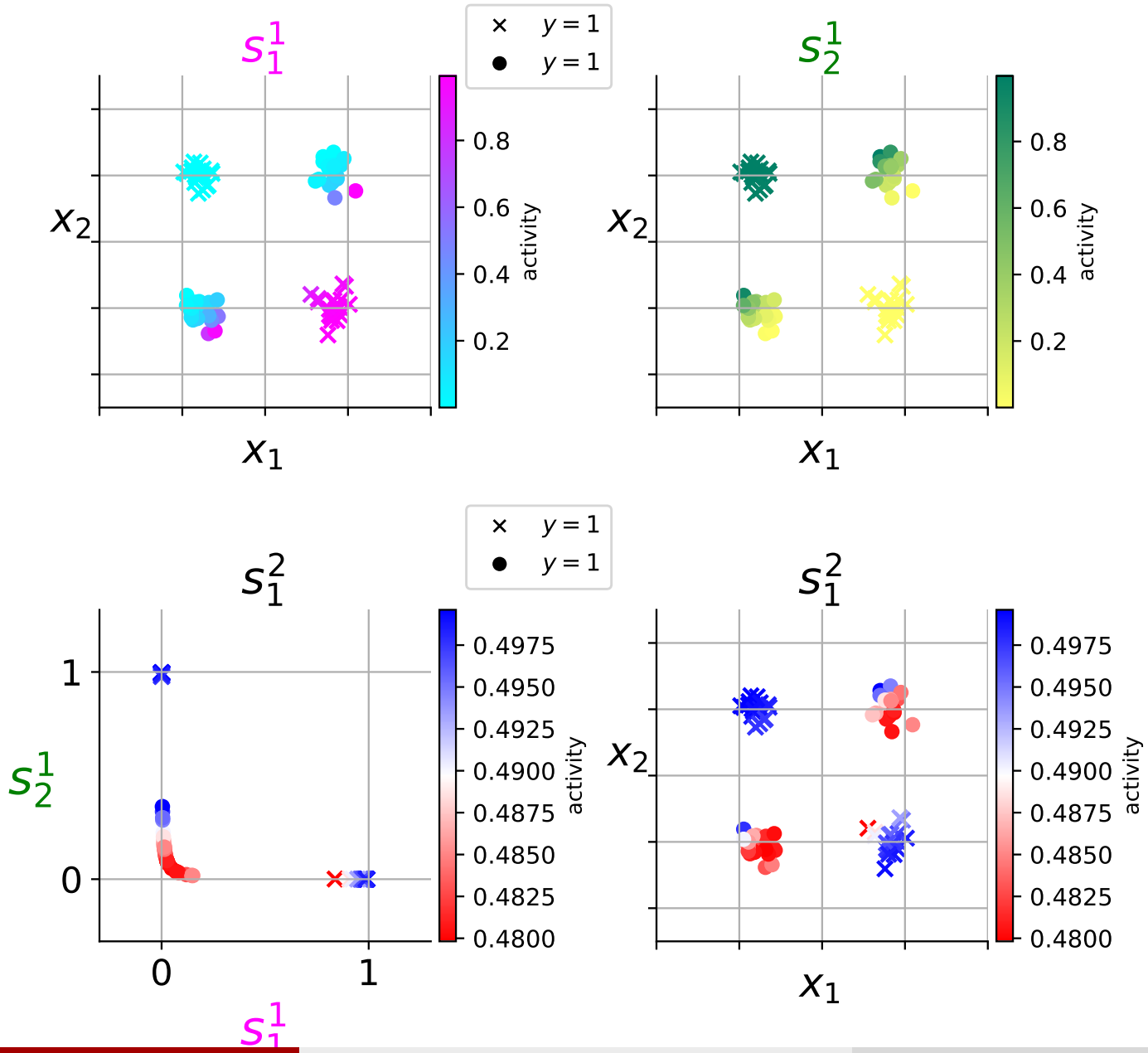
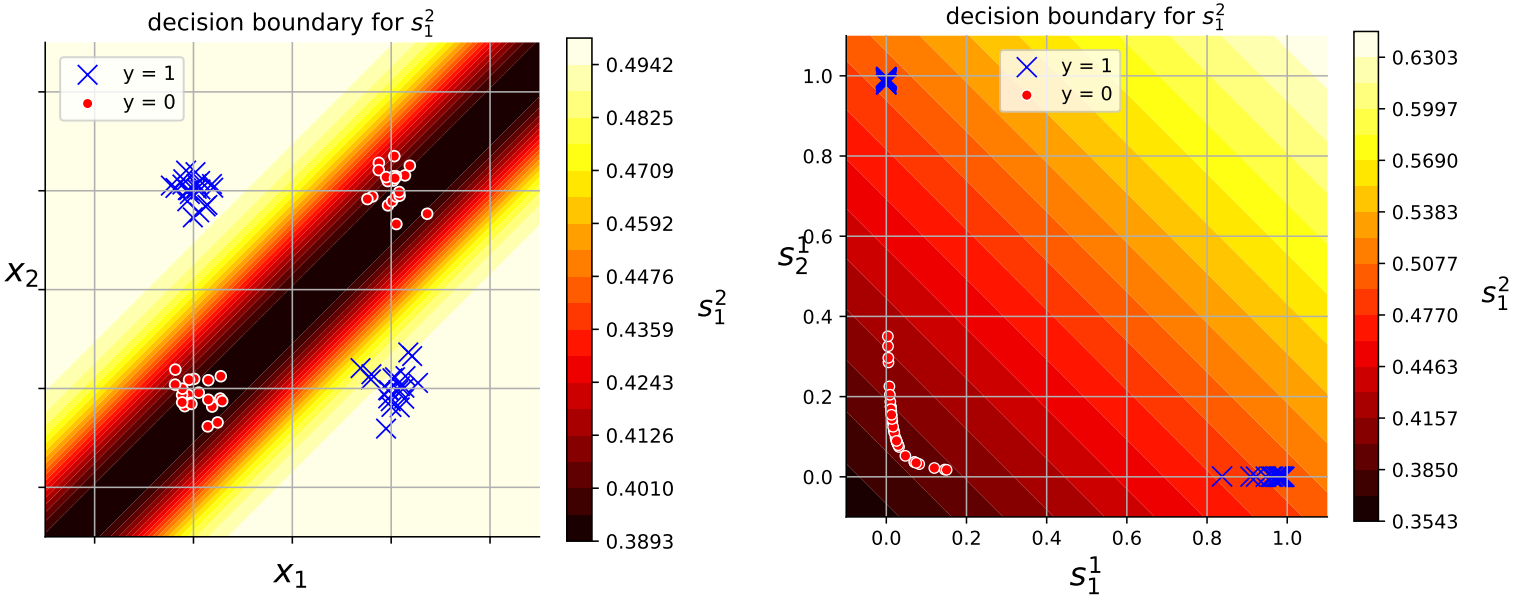


Figure: Solving sub-problems of the XOR problem.

$$\mathrm{XOR}(x_1, x_2) = (\, x_1 \text{ AND } \overline{x}_2 \,) \;\; \text{OR} \;\; (\, \overline{x}_1 \text{ AND } x_2 \,)$$



Non-linear decision boundaries



(a) (b)

Figure: Identifying the decision boundaries.

A new class of models: neural networks

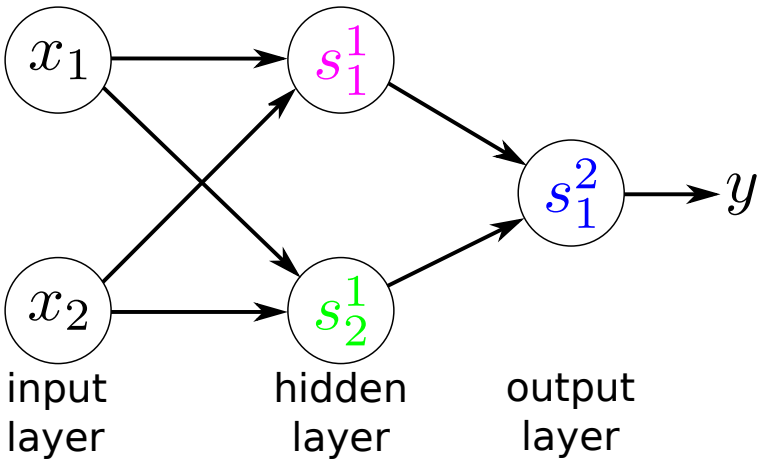


Figure: simplified MLP architecture

Multilayer Perceptrons (MLPs)

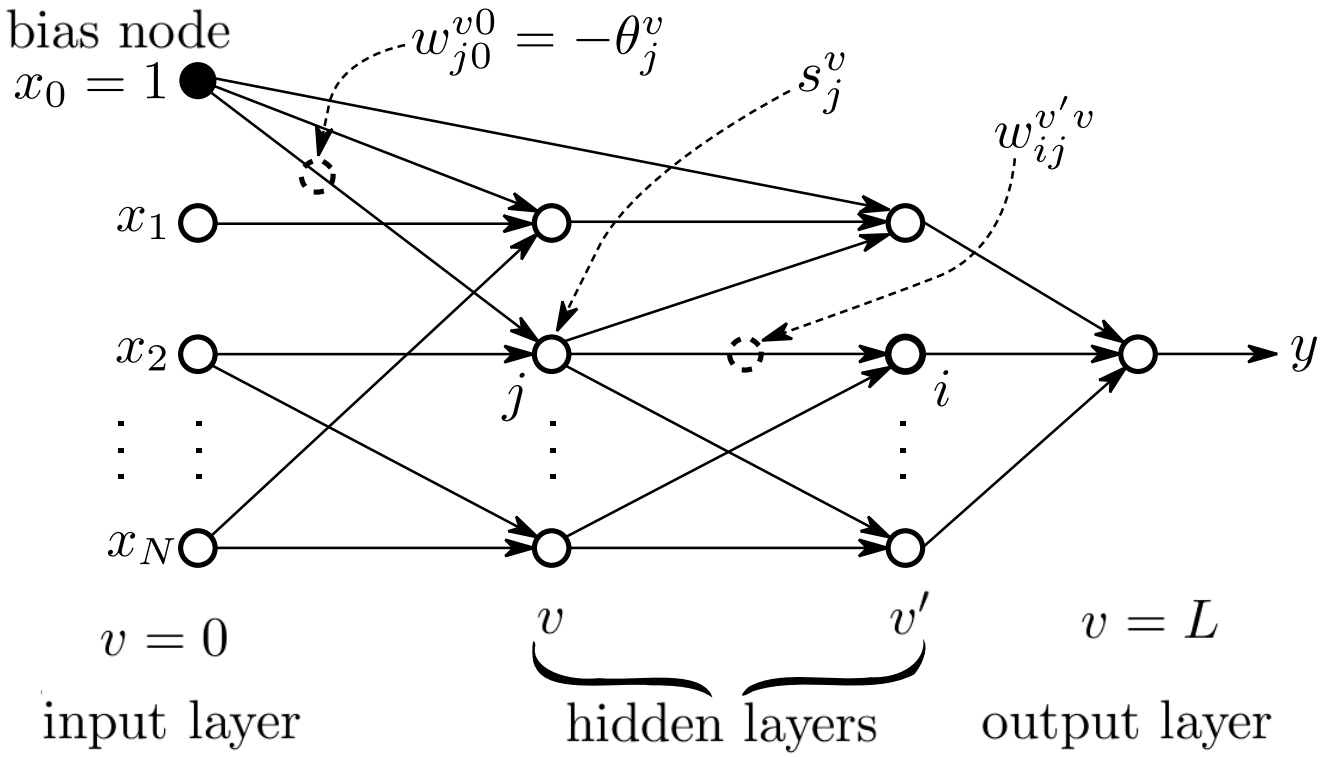
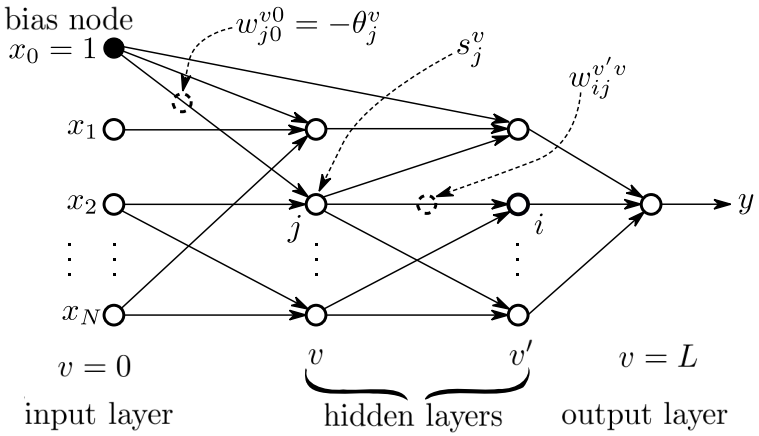
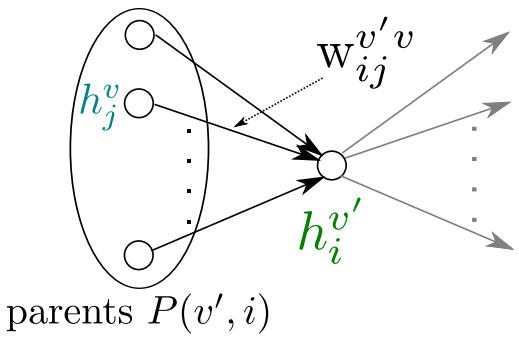


Figure: MLP architecture

$$s_i^{v'} = f_i^{v'}(h_i^{v'})$$

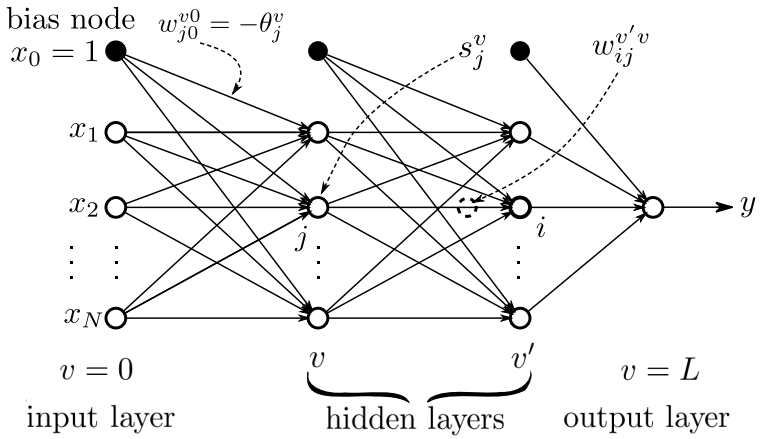
MLP

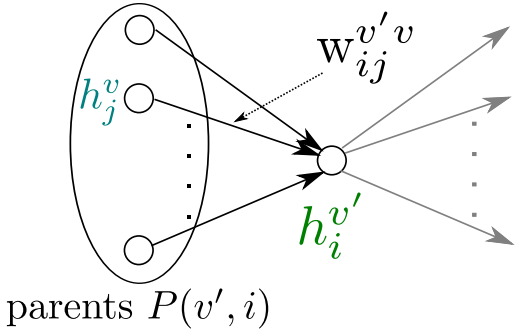




$$h_i^{v'} := \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu} \, f_k^\mu\big(h_k^\mu\big),$$

MLP with **fully connected** layers



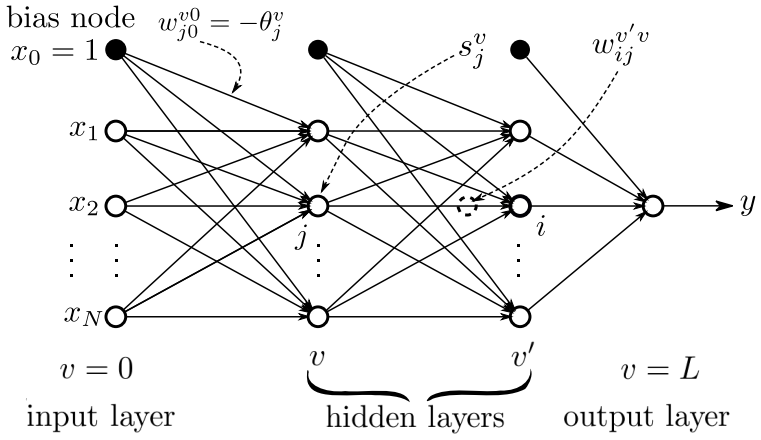


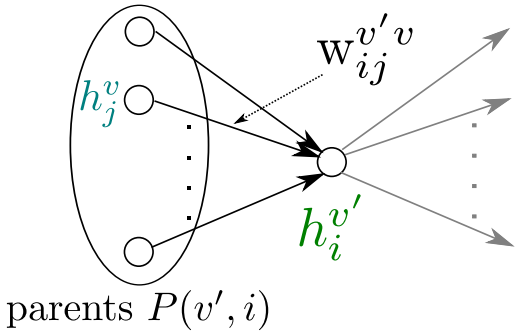
$$h_i^{v'} := \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu}\, f_k^\mu\big(h_k^\mu\big),$$

Full connectivity implies

$$P(v', i) := (\underbrace{s_0^v}_{\substack{=1\\ \text{(bias)}}}, s_1^v, \ldots, s_j^v, \ldots, s_{N_v}^v)^\top,$$

where N_v is the number of hidden nodes in layer v.





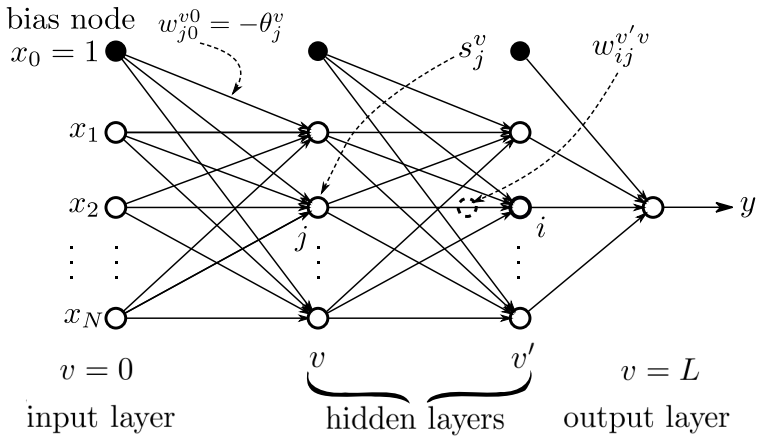
$$h_i^{v'} := \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu}\; f_k^\mu\left(h_k^\mu\right)$$

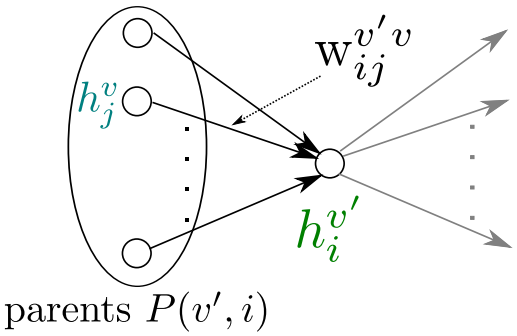
$$h_i^{v'} \overset{\text{fully conn.}}{=} \sum_{j=0}^{N_v} w_{ij}^{v'v}\; f_j^v\left(h_j^v\right)$$

$$= \underline{\mathbf{w}}_i^{v'v\,\top} \cdot \underline{\mathbf{f}}^v\left(\underline{\mathbf{h}}^v\right)$$

$$= \underline{\mathbf{w}}_i^{v'v\,\top} \cdot \underline{\mathbf{s}}^v$$

here $\underline{\mathbf{w}}_i^{v'v} := \left(\overbrace{w_{i0}^{v'v}}^{\text{bias}}, w_{i1}^{v'v}, \ldots, w_{iN_v}^{v'v}\right)^\top \in \mathbb{R}^{N_v+1}$





the total input for all $N_{v'}$ neurons in the layer v

$$\underline{\mathbf{W}}^{v'v} = \begin{pmatrix} | & | & & | & & | \\ \underline{\mathbf{w}}_1^{v'v} & \underline{\mathbf{w}}_2^{v'v} & \cdots & \underline{\mathbf{w}}_i^{v'v} & \cdots & \underline{\mathbf{w}}_{N_{v'}}^{v'v} \\ | & | & & | & & | \end{pmatrix} \in \mathbb{R}^{(N_v+1)\times N_{v'}}$$

Therefore,

$$\underline{\mathbf{h}}^{v'} = \underline{\mathbf{W}}^{v'v\top} \cdot \underline{\mathbf{f}}^v\left(\underline{\mathbf{h}}^v\right) = \underline{\mathbf{W}}^{v'v\top} \cdot \underline{\mathbf{s}}^v$$

Where are we?

Limitations of Perceptrons
- Solving the XOR problem

Feedforward MLP
- Navigating through the indices

Training a neural network
- Recap of gradient-based learning
- Computing gradients for an MLP

The Backpropagation algorithm
- Forward and backward propagation
- Example: MLP with fully connected layers
- Summary of the Backpropagation for gradient descent

Symmetries
- Permutation of indices within consecutive layers
- Sign-reversal across layers

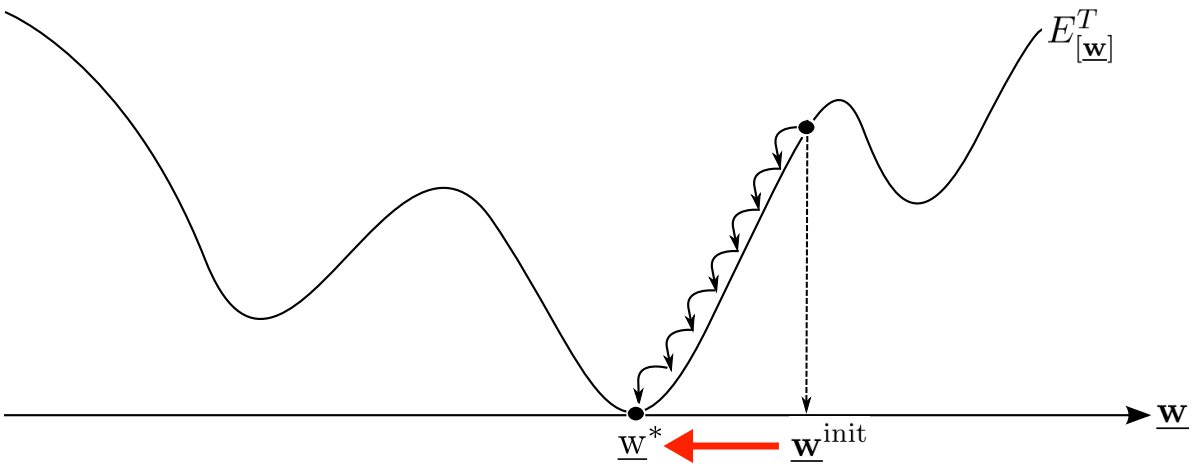
Minimizing the training error

Training error E^T for the training set $\left\{(\underline{\mathbf{x}}^{(\alpha)}, y_T^{(\alpha)})\right\}$:

$$E^T_{[\underline{\mathbf{w}}]} = \frac{1}{p}\sum_{\alpha=1}^{p} e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\right)\right)$$

$$E^T_{[\underline{\mathbf{w}}]} \stackrel{!}{=} \min_{\underline{\mathbf{w}}} \quad \Rightarrow \quad \underline{\mathbf{w}}^* = \operatorname*{argmin}_{\underline{\mathbf{w}}} E^T_{[\underline{\mathbf{w}}]}$$

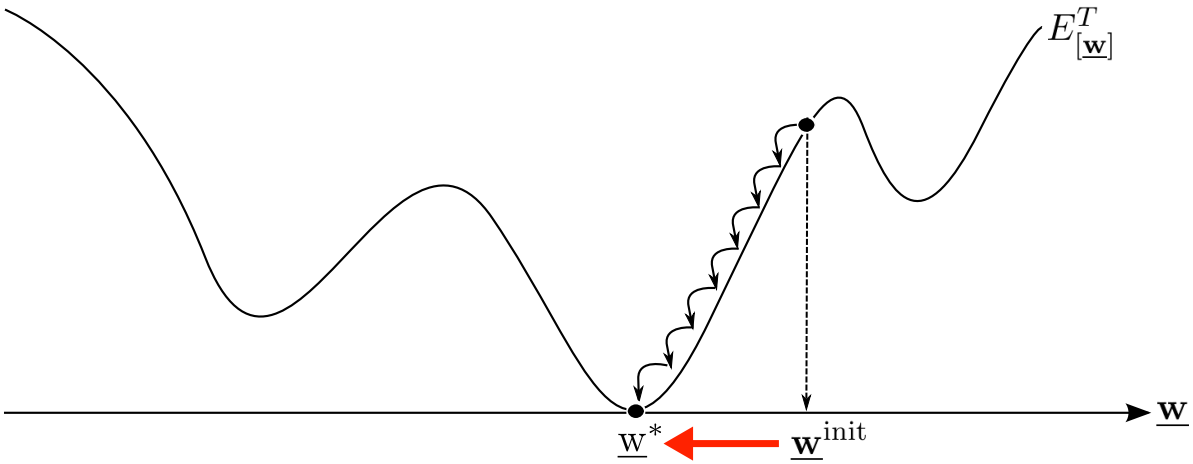
Finding the minimum of $E^T_{[\underline{\mathbf{w}}]}$ iteratively for an MLP



For an MLP:

$$w_{ij}^{v'v}(t+1) \quad = \quad w_{ij}^{v'v}(t) \; - \; \underbrace{\eta}_{\text{learning step}} \cdot \underbrace{\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_{ij}^{v'v}}}_{\text{component of gradient vector}}$$

Finding the minimum of $E^T_{[\underline{\mathbf{w}}]}$ iteratively for an MLP



For an MLP:

$$w_{ij}^{v'v}(t+1) \quad = \quad w_{ij}^{v'v}(t) \; - \; \underbrace{\eta}_{\text{learning step}} \cdot \underbrace{\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_{ij}^{v'v}}}_{\text{component of gradient vector}}$$

Q: *Do you remember why we <u>subtract</u> the gradient from $w_{ij}^{v'v}(t)$?*

Recap: connectionist neuron for minimizing quadratic error

Setting

- $e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right) := \frac{1}{2}\left(y_T - y(\underline{\mathbf{x}}; \underline{\mathbf{w}})\right)^2$
- model class: $y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) := f\left(\sum_{j=0}^{N} w_j x_j\right) = f\left(h(\underline{\mathbf{x}}; \underline{\mathbf{w}})\right)$

$$w_j(t+1) \quad = \quad w_j(t) \; - \; \underbrace{\eta}_{\substack{\text{learning}\\ \text{step}}} \cdot \underbrace{\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_j}}_{\substack{\text{component of}\\ \text{the gradient}}}$$

Recap: connectionist neuron for minimizing quadratic error

Setting

- $e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right) := \frac{1}{2}\left(y_T - y(\underline{\mathbf{x}}; \underline{\mathbf{w}})\right)^2$
- model class: $y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) := f\left(\sum_{j=0}^{N} w_j x_j\right) = f\left(h(\underline{\mathbf{x}}; \underline{\mathbf{w}})\right)$

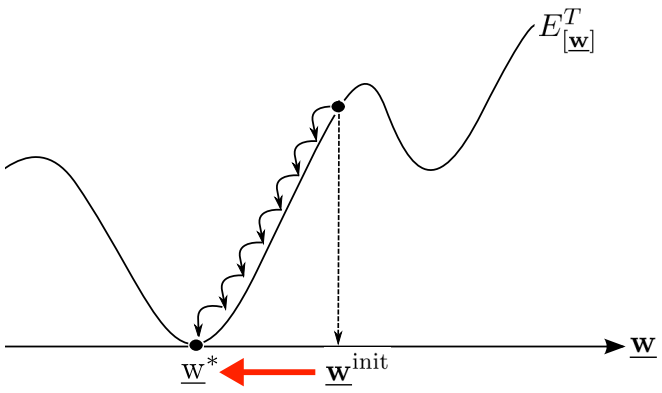
$$\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_j} = \frac{1}{p}\sum_{\alpha=1}^{p} \underbrace{\frac{\partial e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\right)\right)}{\partial y(\underline{\mathbf{x}}^{(\alpha)}, \underline{\mathbf{w}})}}_{\substack{\text{factor depending}\\ \text{on cost function}}} \cdot \underbrace{\frac{\partial y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})}{\partial w_j}}_{\substack{\text{factor depending on}\\ \text{model class}\\ \text{(e.g. perceptron, MLP)}}}$$

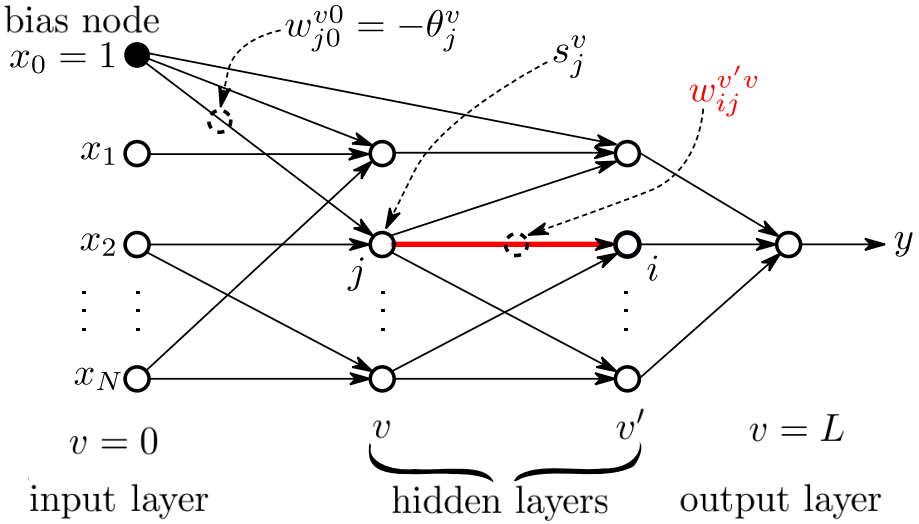
then specifically for quadratic error and the connectionist neuron:

$$= \frac{1}{p}\sum_{\alpha=1}^{p} \left(y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}) - y_T^{(\alpha)}\right) \cdot f'\left(h(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})\right) \cdot x_j$$

Computing gradients for an MLP

$$w_{ij}^{v'v}(t+1) \quad = \quad w_{ij}^{v'v}(t) \; - \; \underbrace{\eta}_{\text{learning step}} \cdot \underbrace{\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_{ij}^{v'v}}}_{\text{component of the gradient vector}}$$





Computing gradients for an MLP

$$w_{ij}^{v'v}(t+1) \quad = \quad w_{ij}^{v'v}(t) \; - \; \underbrace{\eta}_{\substack{\text{learning}\\ \text{step}}} \cdot \underbrace{\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_{ij}^{v'v}}}_{\substack{\text{component of the}\\ \text{gradient vector}}}$$

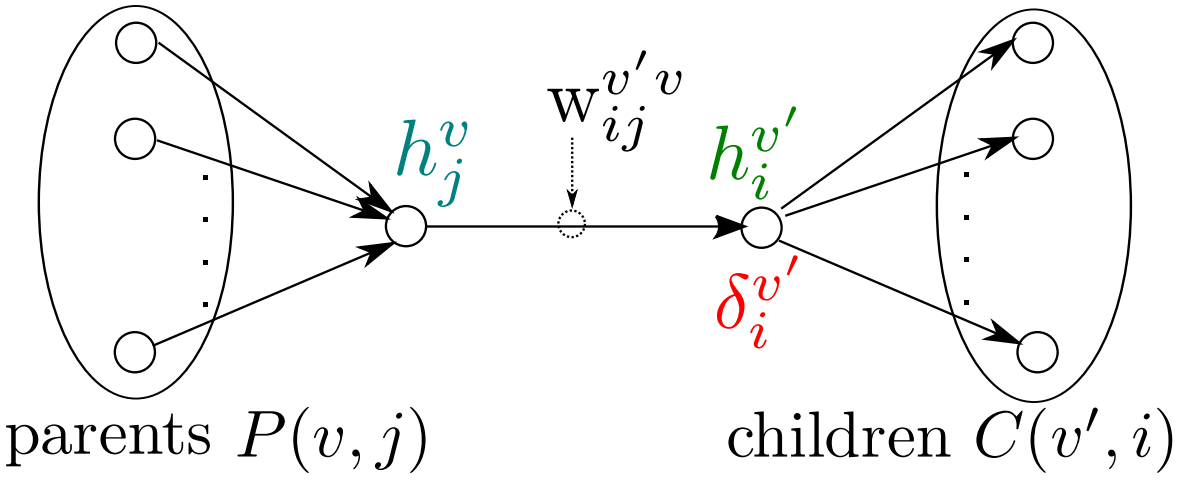
$$\frac{\partial E^T_{[\underline{\mathbf{w}}]}}{\partial w_{ij}^{v'v}} = \frac{1}{p}\sum_{\alpha=1}^{p} \underbrace{\frac{\partial e\left(y_T^{(\alpha)}, y\left(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}}\right)\right)}{\partial y(\underline{\mathbf{x}}^{(\alpha)}, \underline{\mathbf{w}})}}_{\substack{\text{factor depending}\\ \text{on cost function}}} \cdot \underbrace{\frac{\partial y(\underline{\mathbf{x}}^{(\alpha)}; \underline{\mathbf{w}})}{\partial w_{ij}^{v'v}}}_{\substack{\text{factor depending on}\\ \text{model class}\\ \text{(e.g. perceptron, MLP)}}}$$

The Backpropagation algorithm

The Backpropagation algorithm is a method for computing gradients by using the chain rule efficiently.

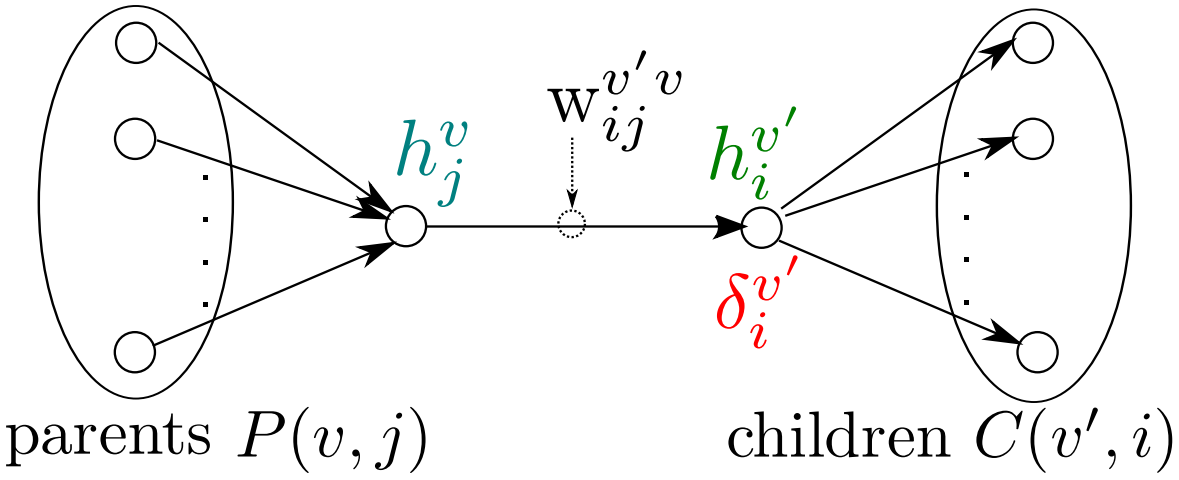
$$\frac{\partial y(\underline{\mathbf{x}}; \underline{\mathbf{w}})}{\partial w_{ij}^{v'v}}$$

Gradients in neural networks



$$h_i^{v'} := \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu}\, f_k^{\mu}(h_k^{\mu}),$$

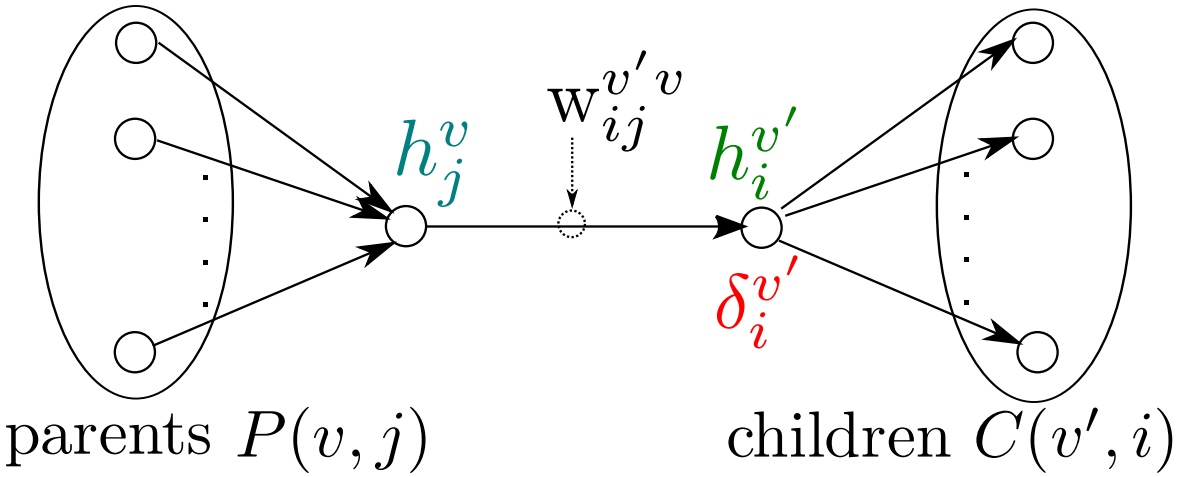
Gradients in neural networks



$$h_i^{v'} := \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu}\, f_k^{\mu}(h_k^{\mu}),$$

Q: *How does* $w_{ij}^{v'v}$ *contribute to the error?* $\frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial w_{ij}^{v'v}} = \ldots$

Gradients in neural networks



$$h_i^{v'} := \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu}\, f_k^{\mu}(h_k^{\mu}),$$

Q: *How does* $w_{ij}^{v'v}$ *contribute to the error?* $\frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial w_{ij}^{v'v}} = \ldots$

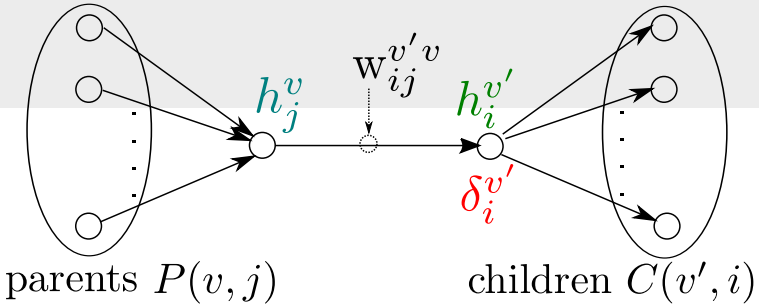
$$\frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial w_{ij}^{v'v}} = \underbrace{\frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial h_i^{v'}}}_{\substack{:=\,\delta_i^{v'} \\ \text{“local error”} \\ \text{at neuron} \\ (v',i)}} \cdot \underbrace{\frac{\partial h_i^{v'}}{\partial w_{ij}^{v'v}}}_{\substack{=\,f_j^{v}(h_j^{v}) \\ \text{activity} \\ \text{of neuron} \\ (v,j)}}$$

The Backpropagation algorithm

The Backpropagation algorithm is composed of two stages:

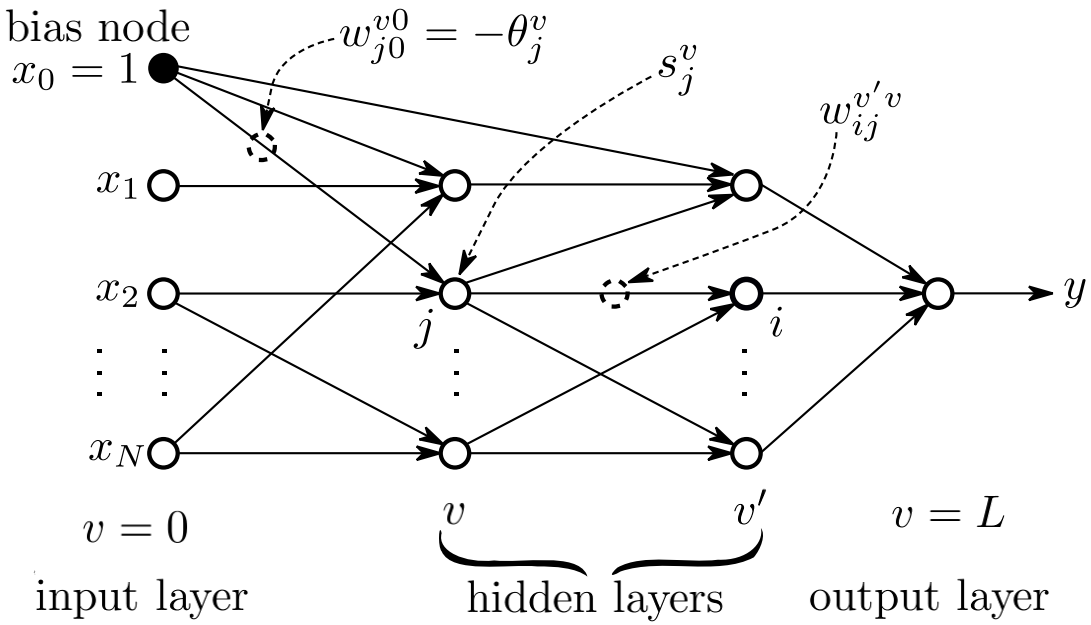
1. **forward propagation** (forward pass)
2. **backward propagation** (backward pass)

The Backpropagation algorithm

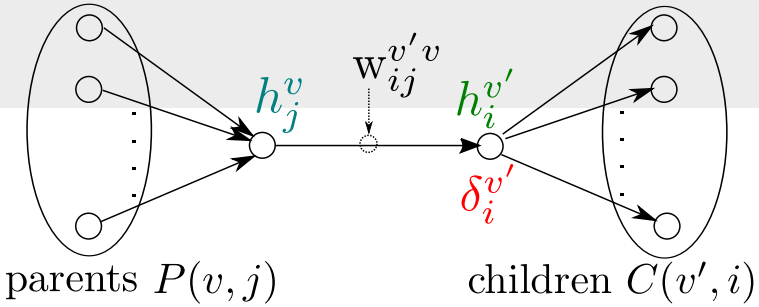


1. **forward propagation**: calculate activities $f_i^{v'}(h_i^{v'})$ (parents → children)

$$h_j^0 := x_j\,, \quad h_i^{v'} = \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu}\, f_k^{\mu}(h_k^{\mu})\,, \quad y(\underline{\mathbf{x}};\underline{\mathbf{w}}) = f_1^L(h_1^L)$$

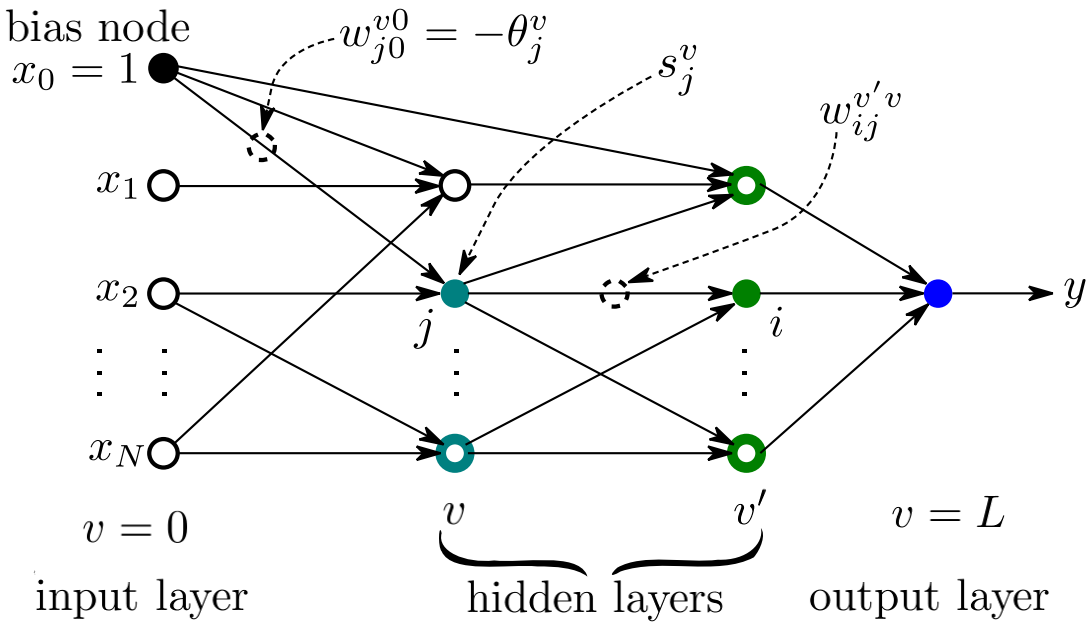


The Backpropagation algorithm

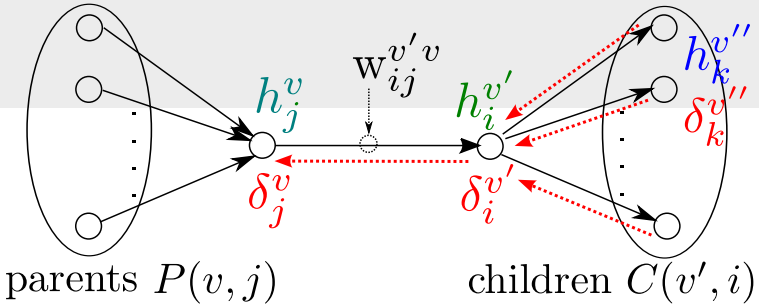


1. **forward propagation**: calculate activities $f_i^{v'}(h_i^{v'})$ (parents → children)

$$h_j^0 := x_j\,, \quad h_i^{v'} = \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu}\, f_k^{\mu}(h_k^{\mu})\,, \quad y(\underline{\mathbf{x}};\underline{\mathbf{w}}) = f_1^L(h_1^L)$$



The Backpropagation algorithm



1. **forward propagation**: calculate activities $f_i^{v'}(h_i^{v'})$ (parents $\to$ children)

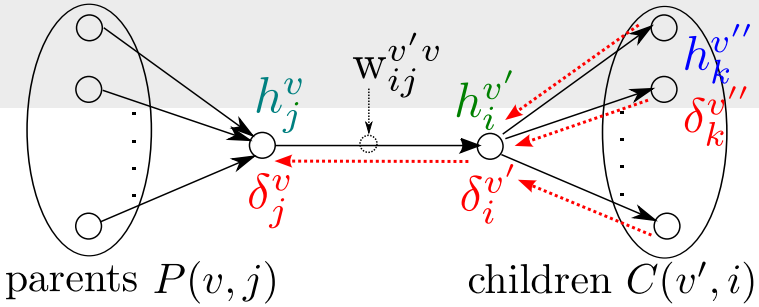
$$h_j^0 := x_j\,, \quad h_i^{v'} = \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu}\, f_k^\mu(h_k^\mu)\,, \quad y(\underline{\mathbf{x}};\underline{\mathbf{w}}) = f_1^L(h_1^L)$$

2. **backpropagation**: calculate "local errors" $\delta_i^{v'}$ (children $\to$ parents)

$$\delta_1^L = \frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial h_1^L} = [f_1^L]'(h_1^L)$$

$$\delta_i^{v'} = \frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial h_i^{v'}} \qquad \text{"local error" at neuron } (v',i)$$

The Backpropagation algorithm



1. **forward propagation**: calculate activities $f_i^{v'}(h_i^{v'})$ (parents $\to$ children)

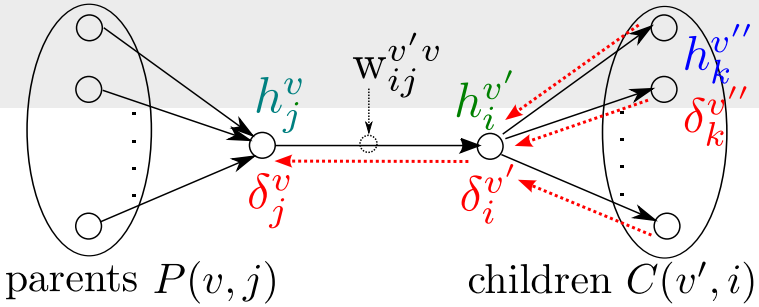
$$h_j^0 := \mathrm{x}_j\,, \quad h_i^{v'} = \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu}\, f_k^{\mu}(h_k^{\mu})\,, \quad y(\underline{\mathbf{x}};\underline{\mathbf{w}}) = f_1^L(h_1^L)$$

2. **backpropagation**: calculate "local errors" $\delta_i^{v'}$ (children $\to$ parents)

$$\delta_1^L = \frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial h_1^L} = [f_1^L]'(h_1^L)$$

$$\delta_i^{v'} = \sum_{(v'',k)\in C(v',i)} \underbrace{\frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial h_k^{v''}}}_{\delta_k^{v''}} \cdot \underbrace{\frac{\partial h_k^{v''}}{\partial h_i^{v'}}}_{w_{ki}^{v''v'}\cdot\,[f_i^{v'}]'(h_i^{v'})} = [f_i^{v'}]'(h_i^{v'}) \sum_{(v'',k)\in C(v',i)} \delta_k^{v''}\, w_{ki}^{v''v'}$$

The Backpropagation algorithm



1. **forward propagation**: calculate activities $f_i^{v'}(h_i^{v'})$ (parents $\to$ children)

$$h_j^0 := x_j\,, \quad h_i^{v'} = \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu}\, f_k^{\mu}(h_k^{\mu})\,, \quad y(\underline{\mathbf{x}};\underline{\mathbf{w}}) = f_1^L(h_1^L)$$

2. **backpropagation**: calculate "local errors" $\delta_i^{v'}$ (children $\to$ parents)

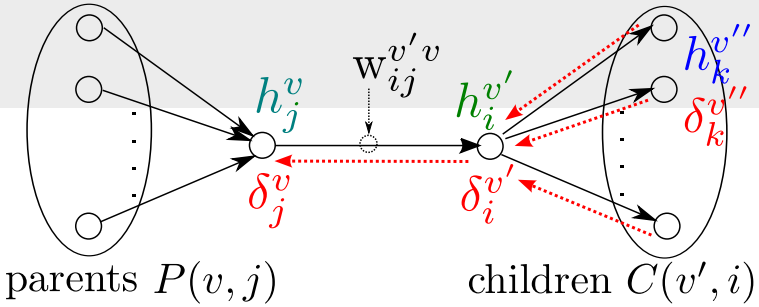
$$\delta_1^L = \frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial h_1^L} = [f_1^L]'(h_1^L)$$

$$\delta_i^{v'} = \sum_{(v'',k)\in C(v',i)} \underbrace{\frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial h_k^{v''}}}_{\delta_k^{v''}} \cdot \underbrace{\frac{\partial h_k^{v''}}{\partial h_i^{v'}}}_{w_{ki}^{v''v'}\cdot [f_i^{v'}]'(h_i^{v'})} = [f_i^{v'}]'(h_i^{v'}) \sum_{(v'',k)\in C(v',i)} \delta_k^{v''} w_{ki}^{v''v'}$$

weight update: using activities and local errors

$$w_{ij}^{v'v} \leftarrow w_{ij}^{v'v} - \eta \cdot \frac{\partial e}{\partial w_{ij}^{v'v}} = w_{ij}^{v'v} - \eta \cdot \frac{\partial e}{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})} \cdot \frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial w_{ij}^{v'v}}$$

The Backpropagation algorithm



1. **forward propagation**: calculate activities $f_i^{v'}(h_i^{v'})$ (parents $\rightarrow$ children)

$$h_j^0 := \mathrm{x}_j\,, \quad h_i^{v'} = \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu}\, f_k^{\mu}(h_k^{\mu})\,, \quad y(\underline{\mathbf{x}};\underline{\mathbf{w}}) = f_1^L(h_1^L)$$

2. **backpropagation**: calculate "local errors" $\delta_i^{v'}$ (children $\rightarrow$ parents)

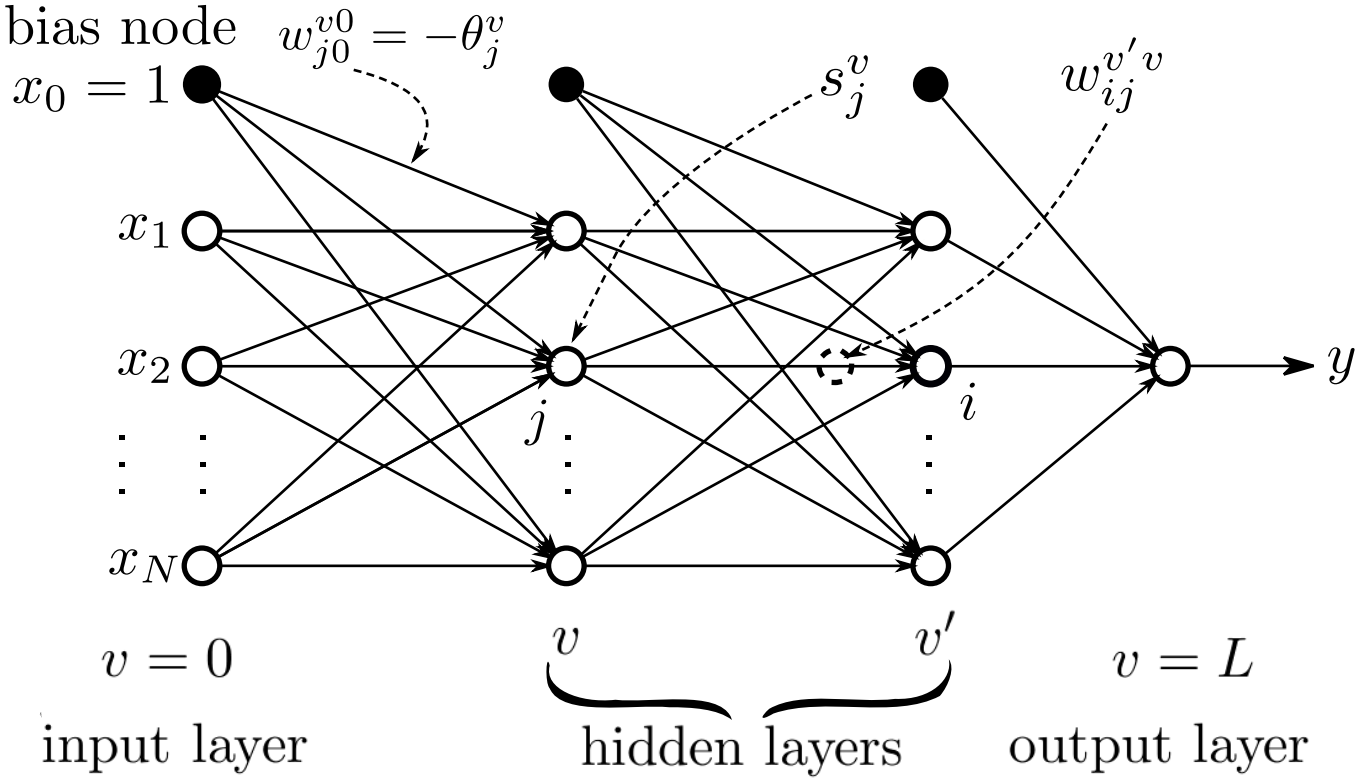
$$\delta_1^L = \frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial h_1^L} = [f_1^L]'(h_1^L)$$

$$\delta_i^{v'} = \sum_{(v'',k)\in C(v',i)} \underbrace{\frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial h_k^{v''}}}_{\delta_k^{v''}} \cdot \underbrace{\frac{\partial h_k^{v''}}{\partial h_i^{v'}}}_{w_{ki}^{v''v'}\cdot\,[f_i^{v'}]'(h_i^{v'})} = [f_i^{v'}]'(h_i^{v'}) \sum_{(v'',k)\in C(v',i)} \delta_k^{v''}\, w_{ki}^{v''v'}$$

weight update: using activities and local errors

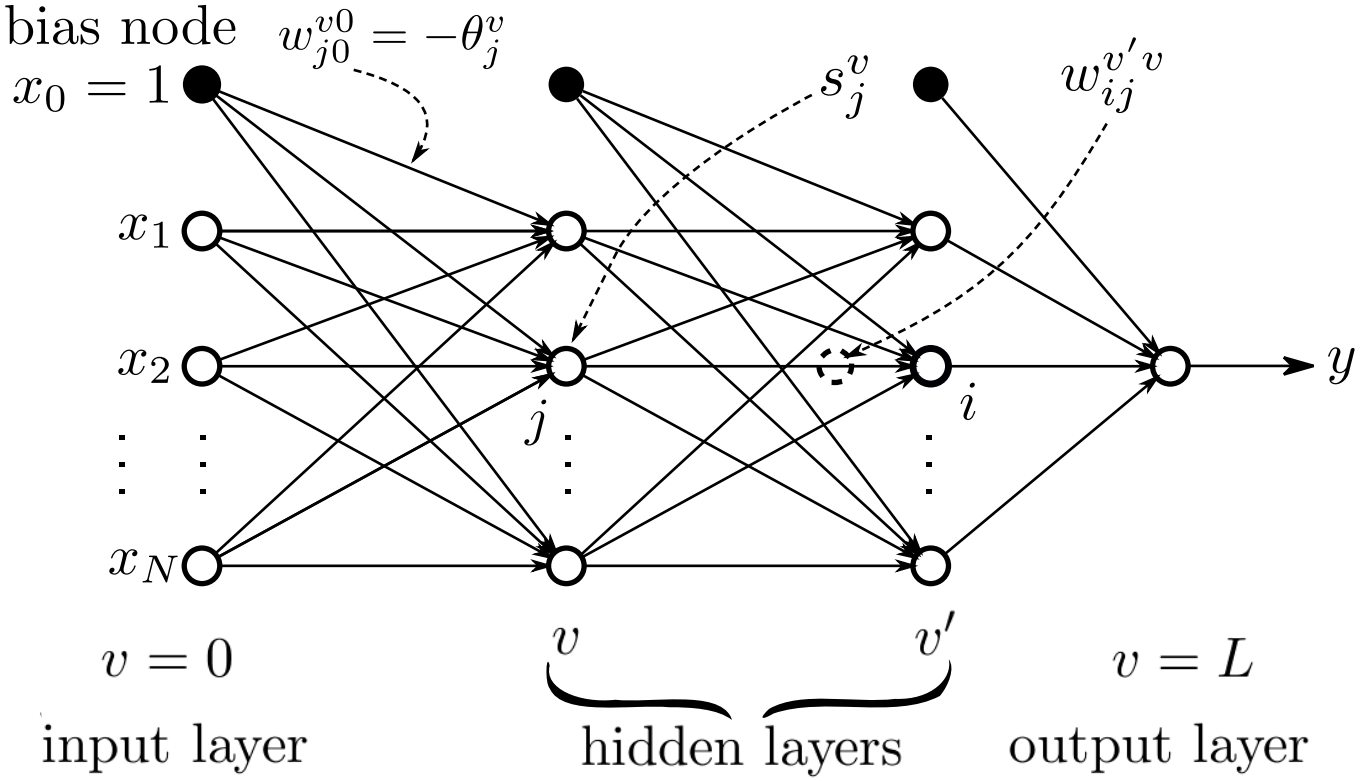
$$w_{ij}^{v'v} \leftarrow w_{ij}^{v'v} - \eta \cdot \frac{\partial e}{\partial w_{ij}^{v'v}} = w_{ij}^{v'v} - \eta \cdot \frac{\partial e}{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})} \cdot \delta_i^{v'} \cdot f_j^v(h_j^v)$$

Example: MLP with fully connected layers



compute the gradients for all weights and biases in this network:

Example: MLP with fully connected layers

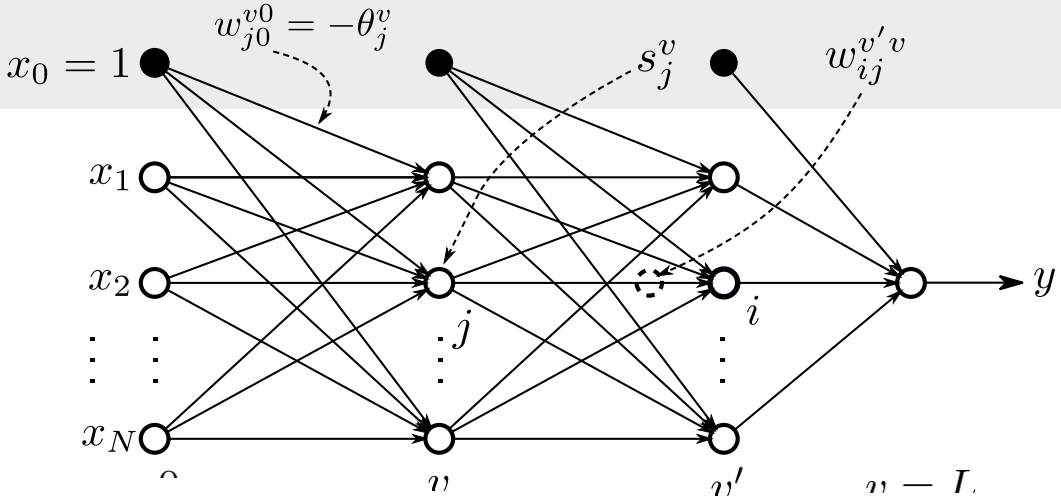


compute the gradients for all weights and biases in this network:

Q: *What is the procedure for the Backpropagation algorithm?*

The forward pass

$$\boxed{h_i^{v'} = \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu} \, f_k^{\mu}\left(h_k^{\mu}\right)}$$

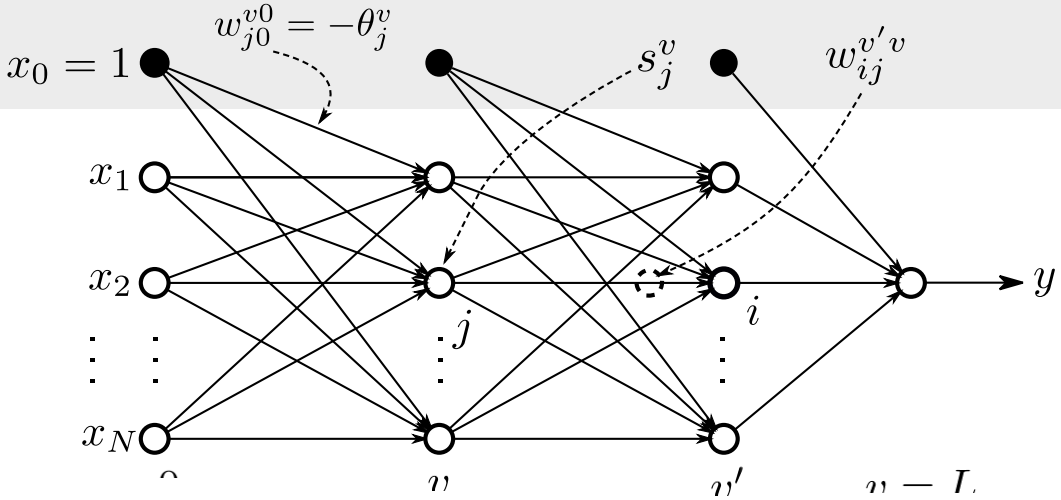


Compute h_i^v for $v = 0, \ldots, L$:

- $v = 0$ (input layer):

$$h_j^0 =$$

The forward pass



$$\boxed{h_i^{v'} = \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu} \, f_k^\mu\big(h_k^\mu\big)}$$

Compute h_i^v :

- $v = 0$ (input layer):

$$h_j^0 = x_j \ , \qquad j = 1, \ldots, N$$

$$h_0^0 = x_0 = 1 \qquad \text{(bias node)}$$

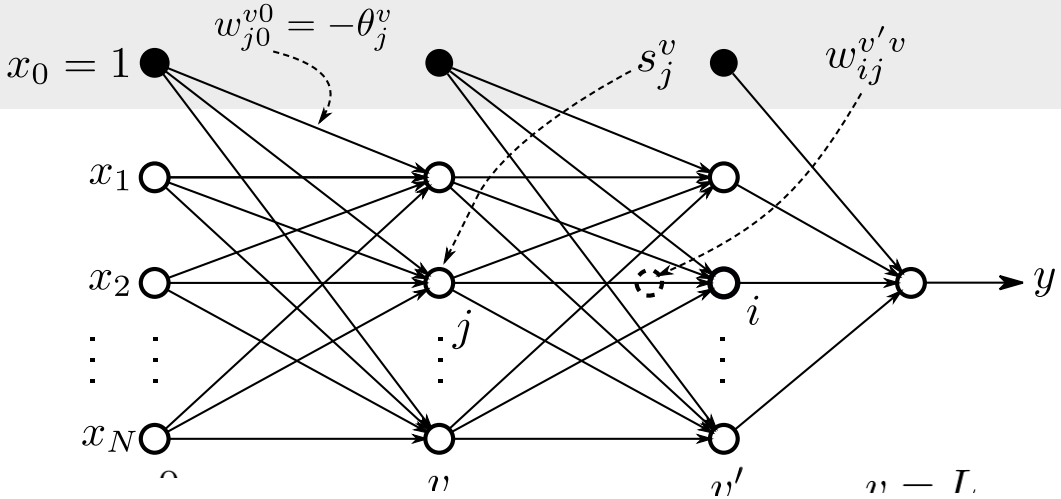
$$\underline{\mathbf{h}}^0 = \underline{\mathbf{x}} \qquad \text{(bias included)}$$

$$f_j^0(h) := h \qquad \text{(identity function)}$$

$$\underline{\mathbf{f}}^0(\underline{\mathbf{h}}) = \underline{\mathbf{x}} \qquad \text{(identity, bias included)}$$

The forward pass

$$\boxed{h_i^{v'} = \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu} \, f_k^{\mu}\left(h_k^{\mu}\right)}$$

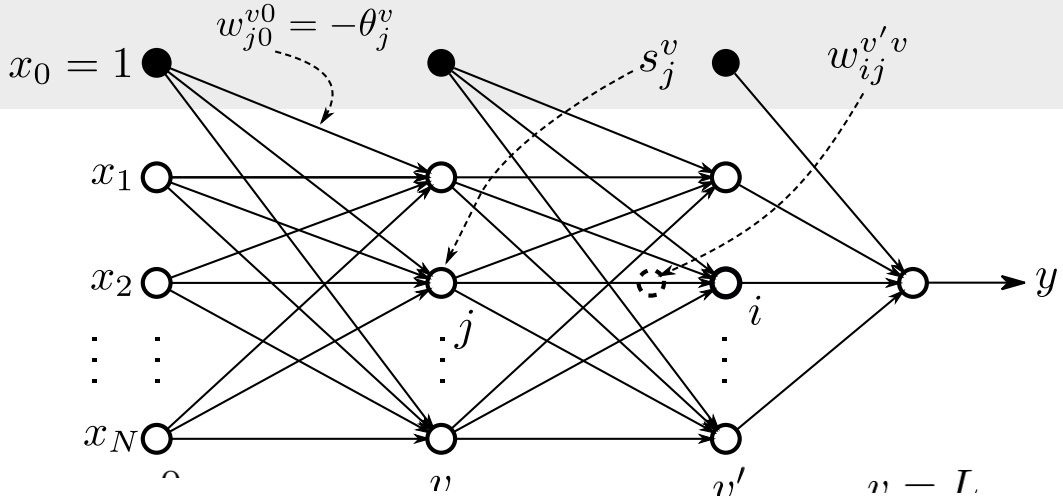


Compute h_i^v :

- $v = 1$ (1st hidden layer with N_1 nodes):

$$h_j^1 =$$

The forward pass



$$\boxed{h_i^{v'} = \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu}\, f_k^\mu\big(h_k^\mu\big)}$$

Compute h_i^v :

- $v = 1$ (1st hidden layer with N_1 nodes):

$$h_j^1 = \sum_{k=0}^{N} w_{jk}^{10}\, f_k^0\big(h_k^0\big) = \underline{\mathbf{w}}_j^{10\top}\underline{\mathbf{h}}^0 \qquad \text{(bias included)}$$

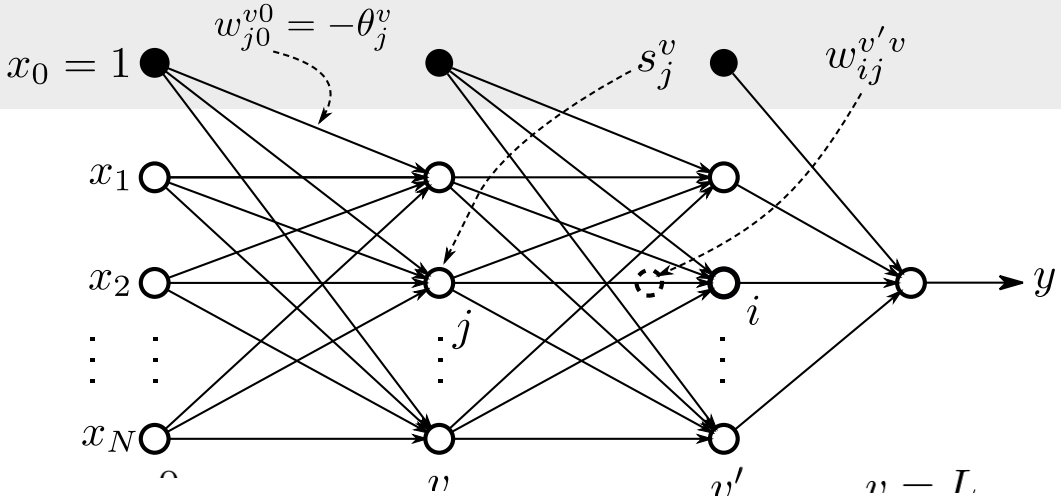
$$\underline{\mathbf{h}}^1 = \underline{\mathbf{W}}^{10\top}\underline{\mathbf{h}}^0 = \underline{\mathbf{W}}^{10\top}\underline{\mathbf{x}} \qquad \text{(biases included)}$$

$$f_j^1(h) := \text{e.g. } \tanh(h), \qquad j = 1, \ldots, N_1$$

$$f_0^1(h) := h \qquad \text{(identity for bias)}$$

The forward pass

$$\boxed{h_i^{v'} = \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu} \, f_k^{\mu}\left(h_k^{\mu}\right)}$$

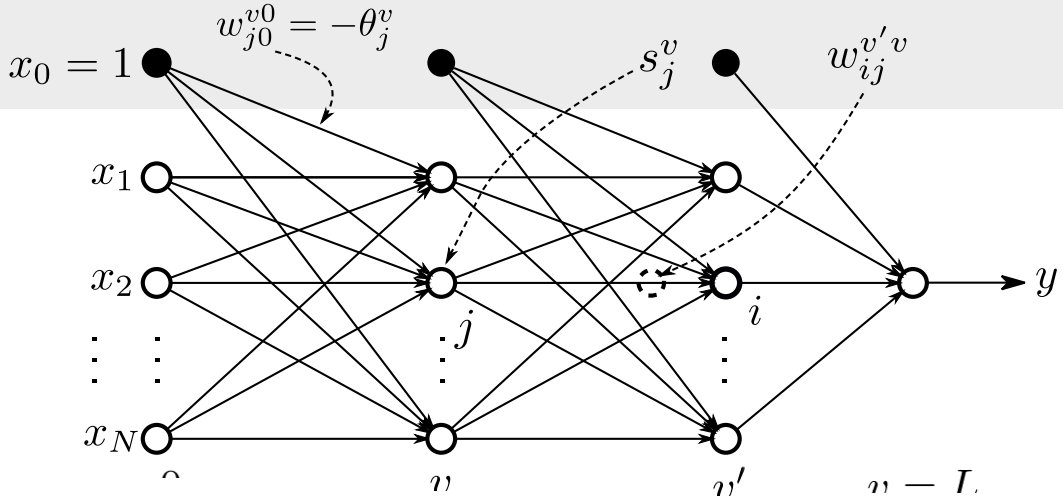


Compute h_i^v :

- $v = 2$ (2nd hidden layer with N_2 nodes):

  $h_i^2 =$

The forward pass



$$\boxed{h_i^{v'} = \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu}\, f_k^{\mu}\big(h_k^{\mu}\big)}$$

Compute h_i^v :

- $v = 2$ (2nd hidden layer with N_2 nodes):

$$h_i^2 = \sum_{j=0}^{N_1} w_{ij}^{21}\, f_j^1\big(h_j^1\big) = \underline{\mathbf{w}}_i^{21\top} \underline{\mathbf{f}}^1(\underline{\mathbf{h}}^1) \qquad \text{(bias included)}$$

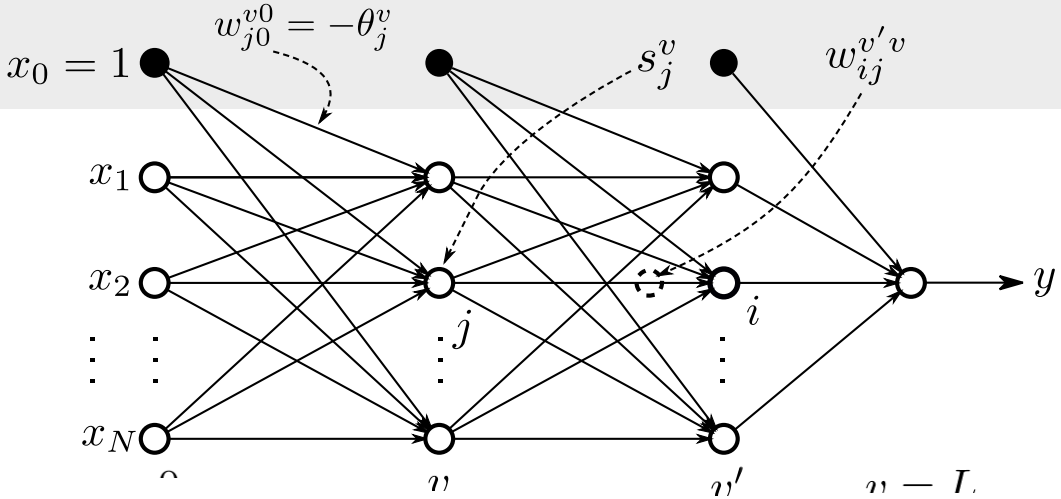
$$\underline{\mathbf{h}}^2 = \underline{\mathbf{W}}^{21\top} \underline{\mathbf{f}}^1(\underline{\mathbf{h}}^1) \qquad \text{(biases included)}$$

$$f_i^2(h) := \text{e.g. } \tanh(h) \qquad i = \mathbf{1}, \ldots, N_2$$

$$f_0^2(h) := h \qquad \text{(identity for bias)}$$

The forward pass

$$\boxed{h_i^{v'} = \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu} \, f_k^{\mu}\left(h_k^{\mu}\right)}$$

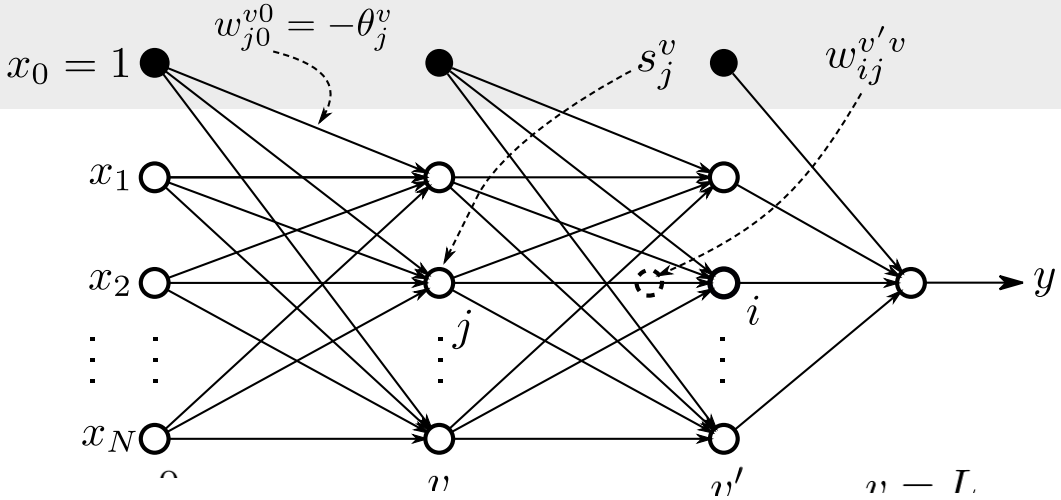


Compute h_i^v :

- $v = L$ (output layer):

$$h_1^L =$$

The forward pass



$$\boxed{h_i^{v'} = \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu}\, f_k^{\mu}\big(h_k^{\mu}\big)}$$

Compute h_i^v :

- $v = L$ (output layer):

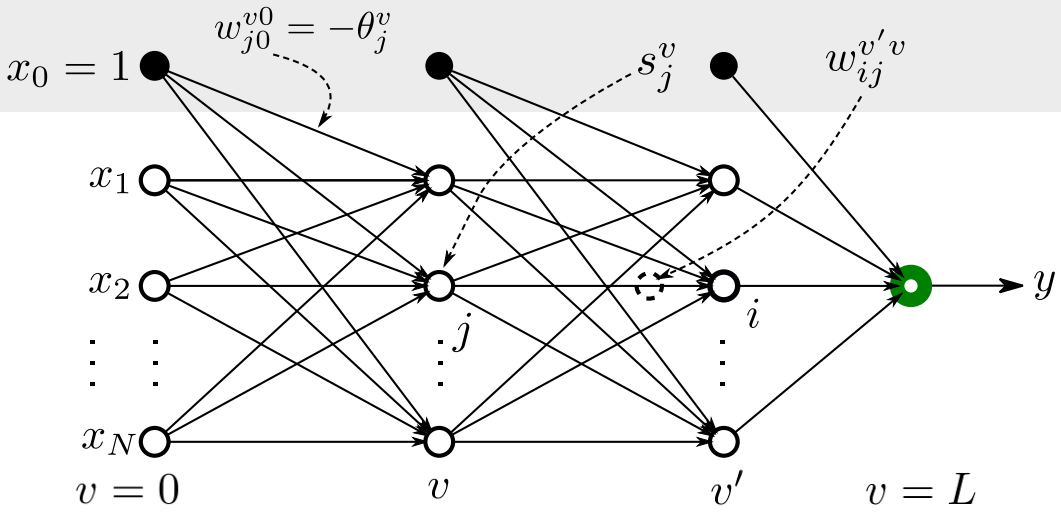
$$h_1^L = \sum_{i=0}^{N_2} w_{1i}^{L2}\, f_i^2\big(h_i^2\big) = \underline{\mathbf{w}}_1^{L2\top}\, \underline{\mathbf{f}}^2(\underline{\mathbf{h}}^2) \qquad \text{(bias included)}$$

$$y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) = f_1^L(h_1^L)$$

$$f_1^L(h) := \text{e.g. logistic function (sigmoid)}$$

The backward pass

$$\boxed{\delta_i^{v'} = [f_i^{v'}]'(h_i^{v'}) \sum_{(v'',k)\in C(v',i)} \delta_k^{v''} w_{ki}^{v''v'}}$$



Compute δ_i^v:

- $v = L$ (output layer):

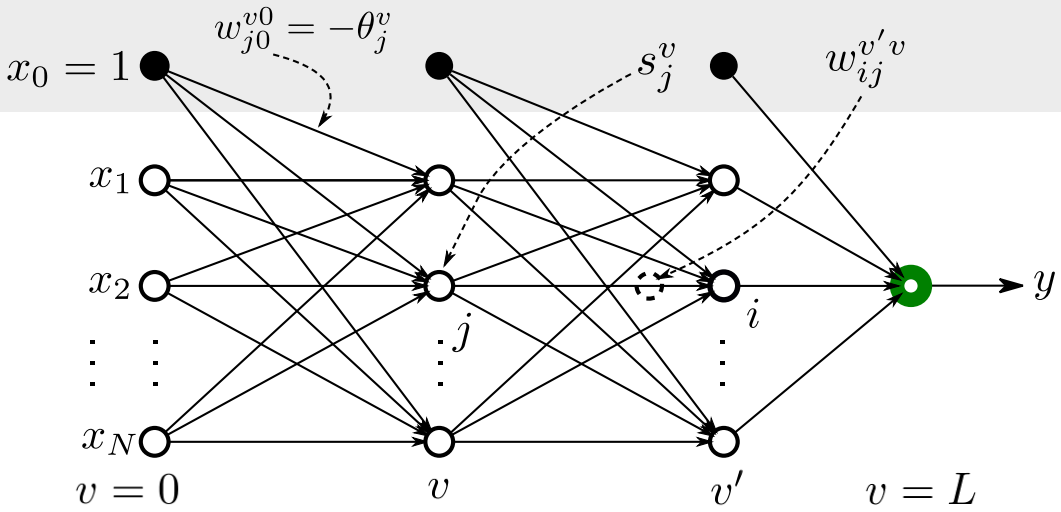
$$y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) = f_1^L(h_1^L)$$

$$f_1^L(h) = \text{e.g. logistic function (sigmoid)}$$

$$\delta_1^L = \frac{\partial y(\underline{\mathbf{x}}; \underline{\mathbf{w}})}{\partial h_1^L} =$$

The backward pass

$$\boxed{\delta_i^{v'} = [f_i^{v'}]'(h_i^{v'}) \sum_{(v'',k)\in C(v',i)} \delta_k^{v''} w_{ki}^{v''v'}}$$



Compute δ_i^v:

- $v = L$ (output layer):

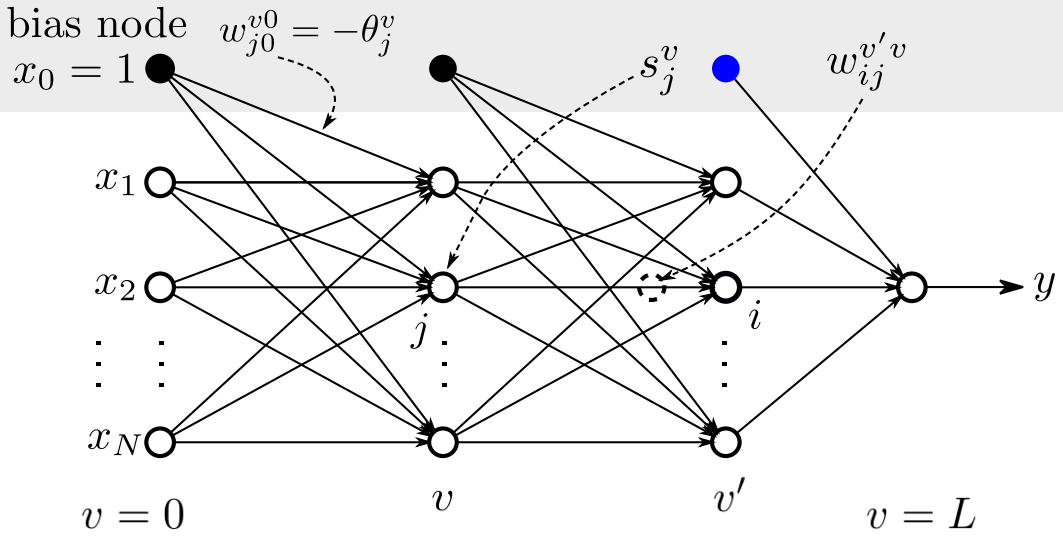
$$y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) = f_1^L(h_1^L)$$

$$f_1^L(h) = \text{e.g. logistic function (sigmoid)}$$

$$\delta_1^L = \frac{\partial y(\underline{\mathbf{x}}; \underline{\mathbf{w}})}{\partial h_1^L} = [f_1^L]'(h_1^L) \cdot 1 \qquad (C(L,1) \text{ is an empty set})$$

The backward pass

$$\boxed{\delta_i^{v'} = [f_i^{v'}]'(h_i^{v'}) \sum_{(v'',k)\in C(v',i)} \delta_k^{v''} w_{ki}^{v''v'}}$$



Compute δ_i^v:

- $v = 2$ (2nd hidden layer with N_2 nodes):

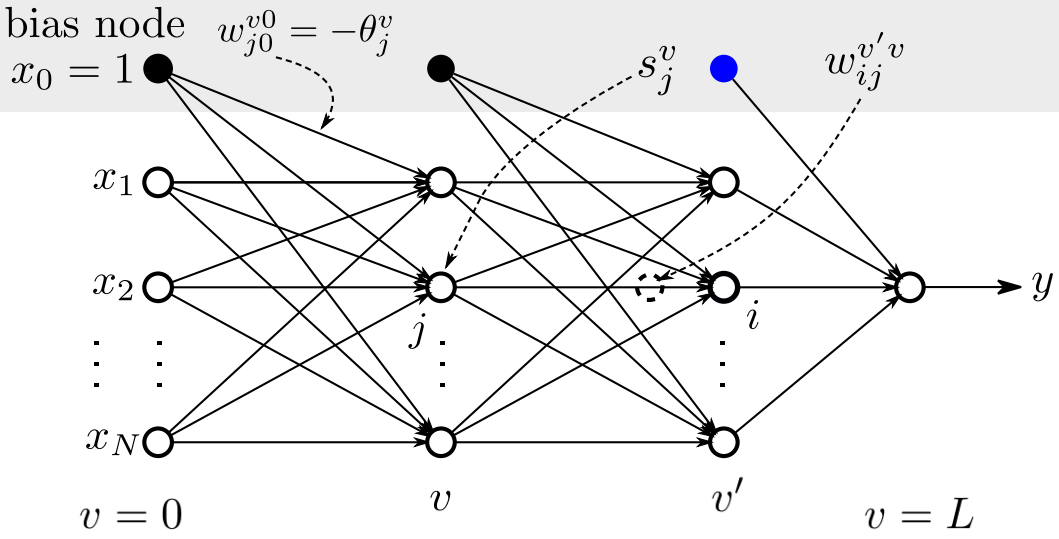
$f_i^2(h) :=$ e.g. $\tanh(h)$, $\qquad i = 1, \ldots, N_2$

$f_0^2(h) := h$ $\qquad$ (identity for bias)

$\delta_i^2 =$

The backward pass

$$\boxed{\delta_i^{v'} = [f_i^{v'}]'(h_i^{v'}) \sum_{(v'',k)\in C(v',i)} \delta_k^{v''} w_{ki}^{v''v'}}$$



Compute δ_i^v:

- $v = 2$ (2nd hidden layer with N_2 nodes):

$$f_i^2(h) := \text{e.g. } \tanh(h), \qquad i = 1, \ldots, N_2$$

$$f_0^2(h) := h \qquad \text{(identity for bias)}$$

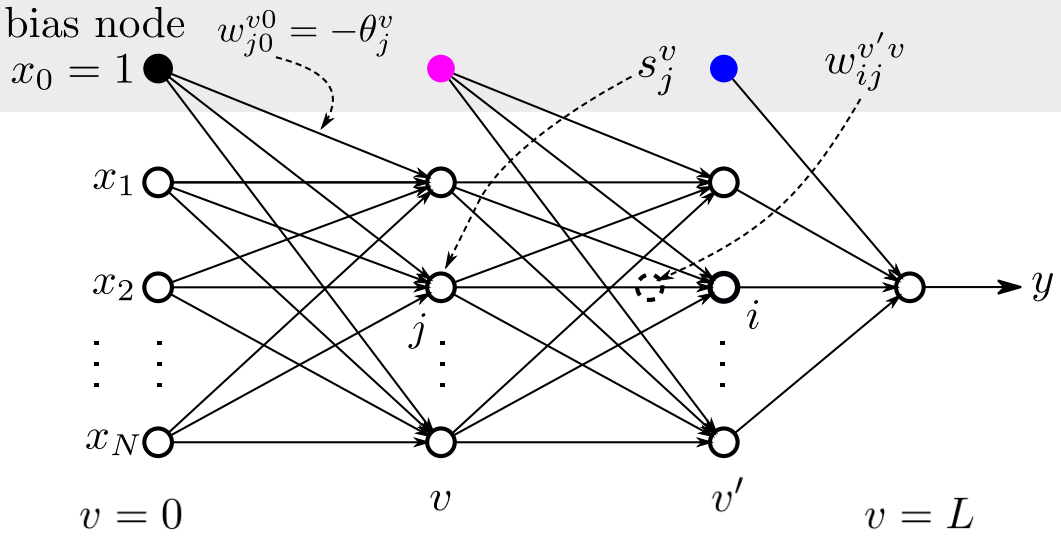
$$\delta_i^2 = [f_i^2]'(h_i^2)\, \delta_1^L\, w_{1i}^{L2}$$

$$\underline{\delta}^2 = [\underline{\mathbf{f}}^2]'(\underline{\widetilde{\mathbf{h}}}^2) \odot (\underline{\widetilde{\mathbf{w}}}_1^{L2} \cdot \delta_1^L)$$

$$\delta_0^2 = 1 \cdot \delta_1^L\, w_{10}^{L2} \qquad \text{(local error for bias)}$$

The backward pass

$$\boxed{\delta_i^{v'} = [f_i^{v'}]'(h_i^{v'}) \sum_{(v'',k)\in C(v',i)} \delta_k^{v''} w_{ki}^{v''v'}}$$



Compute δ_i^v:

- $v = 1$ (1st hidden layer with N_1 nodes):

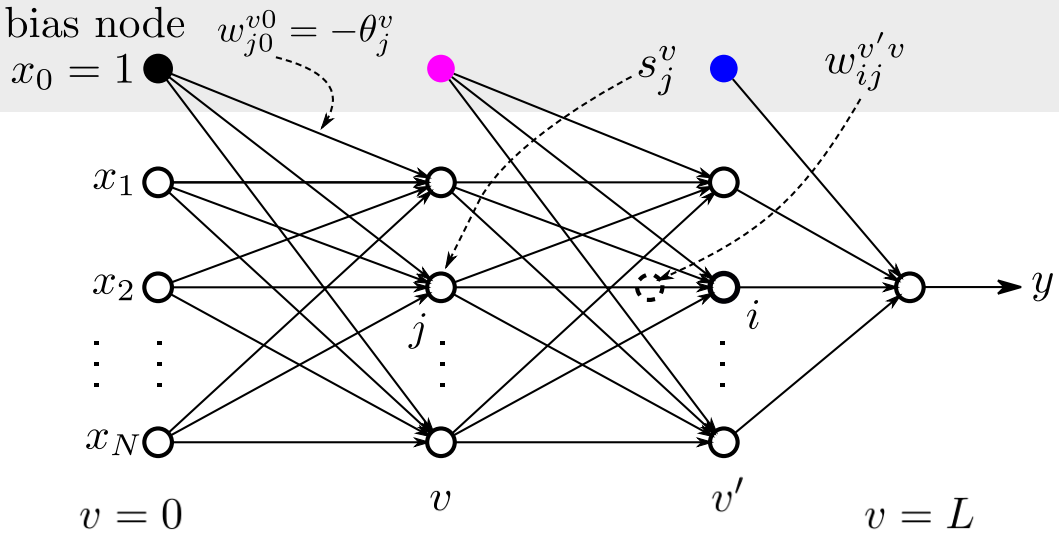
$$f_j^1(h) := \text{e.g. } \tanh(h), \qquad j = 1, \ldots, N_1$$

$$f_0^1(h) := h \qquad \text{(identity for bias)}$$

$$\delta_j^1 =$$

The backward pass

$$\boxed{\delta_i^{v'} = [f_i^{v'}]'(h_i^{v'}) \sum_{(v'',k)\in C(v',i)} \delta_k^{v''} w_{ki}^{v''v'}}$$



Compute δ_i^v:

- $v = 1$ (1st hidden layer with N_1 nodes):

$$f_j^1(h) := \text{e.g. } \tanh(h), \qquad j = 1, \dots, N_1$$

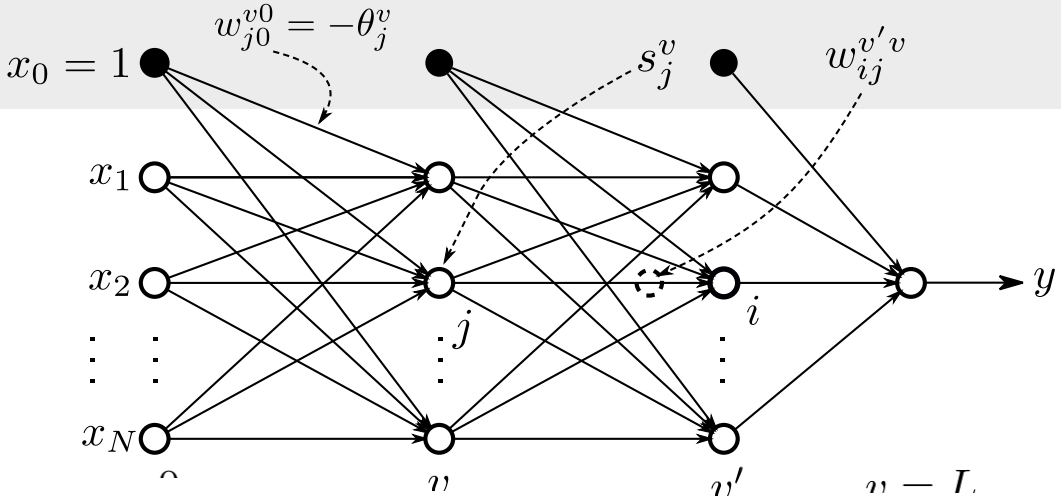
$$f_0^1(h) := h \qquad \text{(identity for bias)}$$

$$\delta_j^1 = [f_j^1]'(h_j^1) \sum_{k=1}^{N_2} \delta_k^2 \, w_{kj}^{21} \qquad \text{(bias } k = 0 \text{ excl.)}$$

$$\underline{\delta}^1 = [\underline{\mathbf{f}}^1]'(\underline{\widetilde{\mathbf{h}}}^1) \odot (\underline{\widetilde{\mathbf{W}}}^{21} \underline{\delta}^2)$$

$$\delta_0^1 = 1 \cdot \sum_{k=1}^{N_2} \delta_k^2 \, w_{k0}^{21} \qquad \text{(local error for bias)}$$

The backward pass



$$\boxed{\delta_i^{v'} = [f_i^{v'}]'(h_i^{v'}) \sum_{(v'',k)\in C(v',i)} \delta_k^{v''} w_{ki}^{v''v'}}$$

Compute δ_i^v:

- $v = 0$ (input layer):

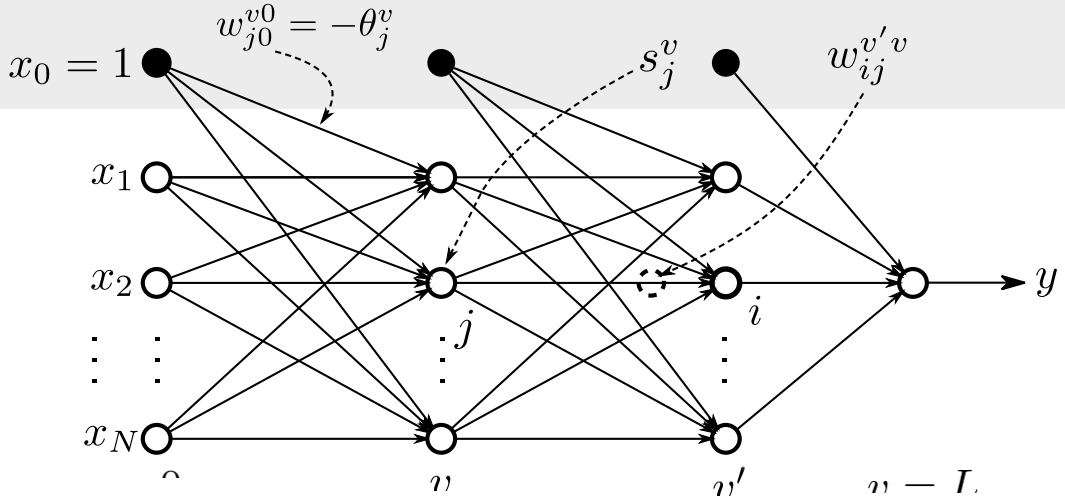
$$f_j^0(h) := h \qquad \text{(identity function)}$$

$$\underline{\mathbf{f}}^0(\underline{\mathbf{h}}) := \underline{\mathbf{x}} \qquad \text{(bias } x_0 \text{ included)}$$

$$\delta_j^0$$

$$\boxed{w_{ij}^{v'v} \leftarrow w_{ij}^{v'v} - \eta \cdot \frac{\partial e}{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})} \cdot \frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial w_{ij}^{v'v}} = w_{ij}^{v'v} - \eta \cdot \frac{\partial e}{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})} \cdot \delta_i^{v'} \cdot f_j^v(h_j^v)}$$

The backward pass



$$\boxed{\delta_i^{v'} = [f_i^{v'}]'(h_i^{v'}) \sum_{(v'',k)\in C(v',i)} \delta_k^{v''} w_{ki}^{v''v'}}$$

Compute δ_i^v:

- $v = 0$ (input layer):

$$f_j^0(h) := h \qquad \text{(identity function)}$$

$$\underline{\mathbf{f}}^0(\underline{\mathbf{h}}) := \underline{\mathbf{x}} \qquad \text{(bias } x_0 \text{ included)}$$

$$\delta_j^0 \qquad \text{(Not applicable to observed input)}$$

$$\boxed{w_{ij}^{v'v} \leftarrow w_{ij}^{v'v} - \eta \cdot \frac{\partial e}{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})} \cdot \frac{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}{\partial w_{ij}^{v'v}} = w_{ij}^{v'v} - \eta \cdot \frac{\partial e}{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})} \cdot \delta_i^{v'} \cdot f_j^v(h_j^v)}$$

Table: Overview of dimensions

layer	parameters	total input	"local" errors
$v = 0$	N/A	$\underline{\mathbf{h}}^0 := \underline{\mathbf{x}}$ $\underline{\mathbf{h}}^0 \in \mathbb{R}^{N+1}$	N/A
$v = 1$	$\underline{\mathbf{w}}_j^{10} \in \mathbb{R}^{N+1}$ $\underline{\mathbf{W}}^{10} \in \mathbb{R}^{(N+1)\times N_1}$	$\underline{\mathbf{h}}^1 \in \mathbb{R}^{N_1+1}$ $\underline{\widetilde{\mathbf{h}}}^1 \in \mathbb{R}^{N_1}$	$\underline{\delta}^1 \in \mathbb{R}^{N_1}$
$v = 2$	$\underline{\mathbf{w}}_i^{21} \in \mathbb{R}^{N_1+1}$ $\underline{\mathbf{W}}^{21} \in \mathbb{R}^{(N_1+1)\times N_2}$ $\underline{\widetilde{\mathbf{W}}}^{21} \in \mathbb{R}^{(N_1)\times N_2}$	$\underline{\mathbf{h}}^2 \in \mathbb{R}^{N_2+1}$ $\underline{\widetilde{\mathbf{h}}}^2 \in \mathbb{R}^{N_2}$	$\underline{\delta}^2 \in \mathbb{R}^{N_2}$
$v = L$	$\underline{\mathbf{w}}_1^{L2} \in \mathbb{R}^{N_2+1}$	$h_1^L \in \mathbb{R}$	$\delta_1^L \in \mathbb{R}$

Summary of the Backpropagation for gradient descent

initialization of weights and thresholds

while *stopping criterion not met* **do**

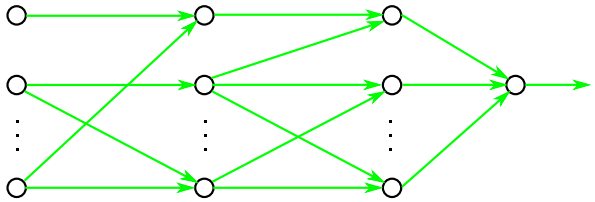
$\quad$ gradient$_{ij}^{v'v} := 0\,, \quad \forall w_{ij}^{v'v}$

$\quad$ **for** $\alpha \in \{1,\dots,p\}$ **do**

$\quad\quad h_i^0 := x_i^{(\alpha)}\,, \quad \forall i$ // forward propagation

$\quad\quad$ **for** $v' \in \{1,\dots,L\}$ **do**

$$\quad\quad\quad h_i^{v'} = \sum_{(\mu,k)\in P(v',i)} w_{ik}^{v'\mu} \underbrace{f_k^\mu(h_k^\mu)}_{x_k^{(\alpha)} \text{ if } v'=1}\,, \quad \forall i$$

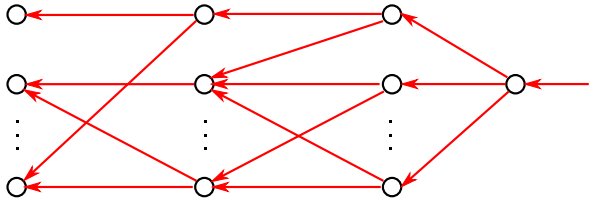


$\quad\quad$ **end**

$\quad\quad \delta_1^L := [f_1^L]'(h_1^L)$ // backward propagation

$\quad\quad$ **for** $v' \in \{L-1,\dots,1\}$ **do**

$$\quad\quad\quad \delta_i^{v'} := [f_i^{v'}]'(h_i^{v'}) \sum_{(\mu,k)\in C(v',i)} \delta_k^\mu\, w_{ki}^{\mu v'}\,, \quad \forall i$$



$\quad\quad$ **end**

$\quad\quad$ gradient$_{ij}^{v'v}$:= gradient$_{ij}^{v'v} + \frac{\partial e^{(\alpha)}}{\partial y(\underline{\mathbf{x}};\underline{\mathbf{w}})}\, \delta_i^{v'}\, f_j^v(h_j^v)\,, \quad \forall w_{ij}^{v'v}$ // sum

$\quad$ **end**

$\quad$ gradient$_{ij}^{v'v}$:= $\frac{1}{p}$gradient$_{ij}^{v'v}$

$\quad w_{ij}^{v'v} := w_{ij}^{v'v} - \eta\,$gradient$_{ij}^{v'v}\,, \quad \forall w_{ij}^{v'v}$ // gradient descent step

end

Summary of the Backpropagation for gradient descent

Complexity

The computational and memory complexity of the Backpropagation algorithm is $\mathcal{O}(n)$, where n is the total number of the weights in the network.
An alternative:

Complexity

The computational and memory complexity of the Backpropagation algorithm is $\mathcal{O}(n)$, where n is the total number of the weights in the network.

An alternative: *finite differences*.

Let $\underline{\widetilde{\mathbf{w}}}$ denote all parameters of the network except for a single weight $w_{ij}^{v'v}$

$$e\big(y_T, y\big(\underline{\mathbf{x}}; \underline{\widetilde{\mathbf{w}}}, w_{ij}^{v'v} \pm \varepsilon\big)\big) =: e\big(\underline{\widetilde{\mathbf{w}}}, w_{ij}^{v'v} \pm \varepsilon\big) \text{ for brevity}$$

Complexity

The computational and memory complexity of the Backpropagation algorithm is $\mathcal{O}(n)$, where n is the total number of the weights in the network.

An alternative: *finite differences*.

Let $\underline{\widetilde{\mathbf{w}}}$ denote all parameters of the network except for a single weight $w_{ij}^{v'v}$

$$e\big(y_T, y\big(\underline{\mathbf{x}}; \underline{\widetilde{\mathbf{w}}}, w_{ij}^{v'v} \pm \varepsilon\big)\big) =: e\big(\underline{\widetilde{\mathbf{w}}}, w_{ij}^{v'v} \pm \varepsilon\big) \text{ for brevity}$$

For $\varepsilon \ll 1$ follows:

$$\frac{\partial e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right)}{\partial w_{ij}^{v'v}} = \frac{e\big(\underline{\widetilde{\mathbf{w}}}, w_{ij}^{v'v} + \varepsilon\big) + e\big(\underline{\widetilde{\mathbf{w}}}, w_{ij}^{v'v} - \varepsilon\big)}{2\,\varepsilon} + \mathcal{O}(\varepsilon^2)$$

Complexity

The computational and memory complexity of the Backpropagation algorithm is $\mathcal{O}(n)$, where n is the total number of the weights in the network.

An alternative: *finite differences*.

Let $\underline{\widetilde{\mathbf{w}}}$ denote all parameters of the network except for a single weight $w_{ij}^{v'v}$

$$e\big(y_T, y\big(\underline{\mathbf{x}}; \underline{\widetilde{\mathbf{w}}}, w_{ij}^{v'v} \pm \varepsilon\big)\big) =: e\big(\underline{\widetilde{\mathbf{w}}}, w_{ij}^{v'v} \pm \varepsilon\big) \text{ for brevity}$$

For $\varepsilon \ll 1$ follows:

$$\frac{\partial e\left(y_T, y\left(\underline{\mathbf{x}}; \underline{\mathbf{w}}\right)\right)}{\partial w_{ij}^{v'v}} = \frac{e\big(\underline{\widetilde{\mathbf{w}}}, w_{ij}^{v'v} + \varepsilon\big) + e\big(\underline{\widetilde{\mathbf{w}}}, w_{ij}^{v'v} - \varepsilon\big)}{2\,\varepsilon} + \mathcal{O}(\varepsilon^2)$$

Complexity of finite differences increases to $\mathcal{O}(n^2)$. But still good for gradient checks.

Symmetries

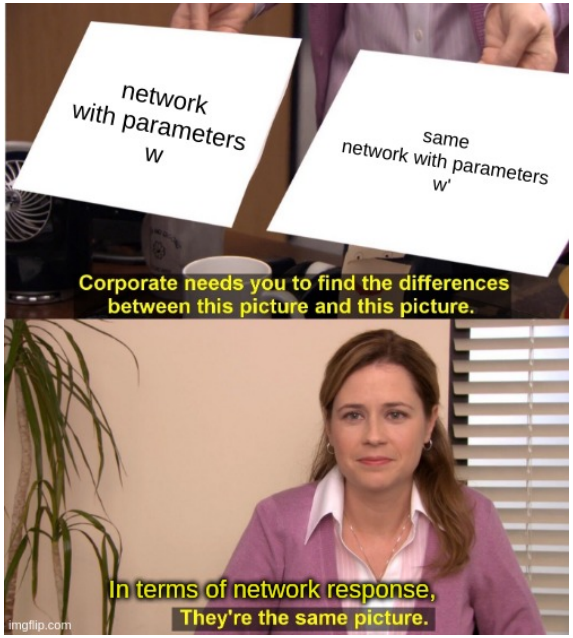
For every $\underline{\mathbf{w}}$ there exists at least one other $\underline{\mathbf{w}}'$ which shares the exact same network response:

$$y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) = y(\underline{\mathbf{x}}; \underline{\mathbf{w}}')$$

Symmetries

For every $\underline{\mathbf{w}}$ there exists at least one other $\underline{\mathbf{w}}'$ which shares the exact same network response:

$$y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) = y(\underline{\mathbf{x}}; \underline{\mathbf{w}}')$$



Permutation of indices within consecutive layers

$$y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) = y(\underline{\mathbf{x}}; \underline{\mathbf{w}}')$$

We can ensure the same model response and training error E^T if:

$$w'^{\,v\,v-1}_{ij} = w^{v\,v-1}_{\Pi(i)j}$$

for some permutation π <u>and</u>

$$w'^{\,v+1\,v}_{ki} = w^{v+1\,v}_{k\pi(i)}$$

then neuron (v, i) behaves identically to neuron $(v, \pi(i))$
$N_v!$ is the total number of possible permutations per hidden layer v.

Sign-reversal across layers

$$y(\underline{\mathbf{x}}; \underline{\mathbf{w}}) = y(\underline{\mathbf{x}}; \underline{\mathbf{w}}')$$

We can ensure the same model response and training error E^T if:

$$\begin{aligned} w'^{\,v\,v-1}_{ij} &= -\,w^{v\,v-1}_{ij}\,, \\ w'^{\,v+1\,v}_{ki} &= -\,w^{v+1\,v}_{ki} \text{ and} \\ f^v(h) = f^{v+1}(h) &:= \tanh(h), \end{aligned}$$

since $\tanh$ is an odd function: $\tanh(-h) = -\tanh(h)$.
2^{N_v} is the total number of sign-reversal combinations per hidden layer v.
Both types of symmetries give us a total of $\prod_{v=1}^{L} N_v!\,2^{N_v}$ eqivalent solutions